A Generalized Adjusted Min-Sum Decoder for 5G LDPC Codes: Algorithm and Implementation

Yuqing Ren, *Student Member, IEEE,* Hassan Harb, *Member, IEEE,* Yifei Shen, *Member, IEEE,* Alexios Balatsoukas-Stimming, *Member, IEEE,* and Andreas Burg, *Senior Member, IEEE*

***Abstract*—5G New Radio (NR) has stringent demands on both performance and complexity for the design of low-density parity-check (LDPC) decoding algorithms and corresponding VLSI implementations. Furthermore, decoders must fully support the wide range of all 5G NR blocklengths and code rates, which is a significant challenge. In this paper, we present a high-performance and low-complexity LDPC decoder, tailor-made to fulfill the 5G requirements. First, to close the gap between belief propagation (BP) decoding and its approximations in hardware, we propose an extension of adjusted min-sum decoding, called generalized adjusted min-sum (GA-MS) decoding. This decoding algorithm flexibly truncates the incoming messages at the check node level and carefully approximates the non-linear functions of BP decoding to balance the error-rate and hardware complexity. Numerical results demonstrate that the proposed fixed-point GA-MS has only a minor gap of 0.1 dB compared to floating-point BP under various scenarios of 5G standard specifications. Secondly, we present a fully reconfigurable 5G NR LDPC decoder implementation based on GA-MS decoding. Given that memory occupies a substantial portion of the decoder area, we adopt multiple data compression and approximation techniques to reduce 42.2% of the memory overhead. The corresponding 28nm FD-SOI ASIC decoder has a core area of 1.823 mm^2 and operates at 895 MHz. It is compatible with all 5G NR LDPC codes and achieves a peak throughput of 24.42 Gbps and a maximum area efficiency of 13.40 Gbps/mm^2 at 4 decoding iterations.**

***Index Terms*—LDPC codes, generalized adjusted min-sum (GA-MS) decoding, belief propagation (BP), hardware implementation, 5G NR wireless communications.**

I. Introduction

LOW-DENSITY parity-check (LDPC) codes, invented by Gallager [1], have received considerable attention in both academia and industry owing to their extraordinary error-correcting performance and the inherently parallel decoding algorithm. Over the past several decades, LDPC codes have been adopted by various communication and storage systems, such as ATSC [2], IEEE 802.11n [3], and DVB-S2 [4]. Most prominently, LDPC codes were ratified as the channel coding scheme of the enhanced mobile broadband (eMBB) scenario in 5G standards [5], [6]. However, designing high-performance and low-complexity LDPC decoding algorithms and corresponding VLSI implementations tailored to 5G New Radio (NR) is still an important research challenge.

In terms of decoding algorithms, belief propagation decoding (also called sum-product (SP) decoding on factor graphs [7]) of LDPC codes delivers outstanding error-correcting performance, closely approaching the Shannon limit [8]. However, SP decoding comes with high computational complexity and memory overhead [9]. To alleviate these issues, a series of min-sum (MS) decoding algorithms [10]–[18] and a series of approximate-min* (A-Min*) decoding algorithms [19], [20] are proposed as alternatives to SP decoding. For instance, MS decoding only involves selecting the smallest magnitudes among incoming messages from variable nodes (VNs) to check nodes (CNs), which simultaneously reduces the memory of outgoing messages from CNs to VNs. However, this process results in performance loss compared to SP decoding [20], [21]. Therefore, more advanced MS variants have been presented [12]–[18], such as normalized MS (NMS) decoding, offset MS (OMS) decoding, adaptive MS (AMS) decoding [13]–[15], self-correction MS decoding [16], and multiple dimensional modified MS decoding [17], [18].

On the other hand, A-Min* decoding first determines the edge with the smallest incoming message, then calculates two distinct outgoing magnitudes at each CN, and propagates them to adjacent VNs, relying on identical SP functions [19]. While this method nearly matches the performance of SP decoding, A-Min* decoding suffers from two significant drawbacks: long decoding latency from the above sequential processing and substantial computational complexity due to the SP functions. To mitigate the decoding latency issue, the authors of [20] proposed a generalized A-Min* (GA-Min*) decoding algorithm by truncating the number of incoming messages to optimize recursive CN processing. To reduce computational complexity, adjusted MS (A-MS) decoding proposed by Qualcomm [21] has drawn significant attention during the development of 5G NR LDPC codes. A-MS decoding can be considered as a quantized version of A-Min* decoding, as it employs look-up tables (LUTs) to simplify non-linear CN processing. However, compared to classical MS-based decoders [22]–[25], A-MS decoding still faces a relatively high implementation complexity due to extra additions and comparisons in the approximation process.

In terms of hardware implementations and given that the 5G standard stipulates a peak throughput of 20 Gbps in the downlink [26], the 5G NR LDPC decoder is tasked with balancing throughput, area efficiency, and energy consumption. This balance must also uphold compatibility across all code configurations, presenting a significant challenge. Notably, since the large amount of memory required to support the maximum blocklength in 5G NR already occupies a significant part of the decoder area, these large memories tend to reduce the impact of more complex algorithms on overall efficiency. In the literature [9], [22]–[25], [27]–[29], numerous classical LDPC decoders have been presented, featuring varying degrees of resource sharing. These decoders can be categorized into fully-parallel [9], [27]–[29] and partially-

Y. Ren, H. Harb, Y. Shen, and A. Burg are with the Telecommunications Circuits Laboratory (TCL), École Polytechnique Fédérale de Lausanne (EPFL), Lausanne 1015, Switzerland (email: {yuqing.ren, hassan.harb, yifei.shen, andreas.burg}@epfl.ch). *Corresponding author: Andreas Burg*.

A. Balatsoukas-Stimming is with the Department of Electrical Engineering, Eindhoven University of Technology, 5600 MB Eindhoven, The Netherlands (email: a.k.balatsoukas.stimming@tue.nl).

parallel architectures [22]–[25]. Partially-parallel decoders can be further divided into block-parallel, row-parallel, and other variants. It is noteworthy that thanks to the quasi-cyclic (QC) property of the LDPC codes in many standards [3], [5], [6], block-parallel architectures can implement LDPC decoding in an iteratively decomposed fashion without high routing complexity, resulting in a balance between throughput, area efficiency, and decoding flexibility [24], [25].

To adhere to the 5G peak throughput requirement, several state-of-the-art (SOA) 5G NR LDPC decoders have been reported in [15], [30]–[35], mainly using row-parallel architectures or variations thereof. Unlike block-parallel ones, these row-parallel architectures can process multiple blocks simultaneously by a more complex programmable routing network to improve peak throughput. However, due to the high routing complexity for long codes, it is difficult for these designs to support the maximum blocklength in 5G NR. Thus, the implementations in [15], [32]–[35] generally target or provide results only for short to moderate blocklengths. Only recently such a 5G NR LDPC decoder with a novel memory access scheduling [31] was reported to fully meet the requirements of 5G. Based on partially row-parallel (PRP) architecture, the decoder in [31] can process each layer of the 5G NR LDPC base graphs within a predetermined fixed latency to achieve a high peak throughput. Nevertheless, the row weights of the base graphs vary significantly, and most layers have low weights [5], [6]. During decoding at medium to low code rates, the efficiency of this PRP architecture [31] is limited by its row-parallel design that must still process layers with low weights sequentially at a fixed rate, thus diminishing the decoding throughput. Yet, considering the wide range of 5G NR LDPC blocklengths and code rates, the block-parallel architecture has an inherent advantage in balancing flexibility and parallelism, achieving stable performance across all 5G NR code configurations. Our work demonstrates that a block-parallel architecture can already satisfy the 5G peak throughput requirement of 20 Gbps, without higher processing parallelism.

Contributions and Paper Outline:

The specific contributions of this paper are as follows:

- We propose a high-performance and low-complexity algorithm called generalized A-MS (GA-MS) decoding to balance the error-rate, computational complexity, and memory overhead. We also design the required LUTs and propose other optimizations to simplify the hardware.
- We provide a comprehensive performance analysis with the designed LUTs, quantization schemes, and approximation techniques. The proposed fixed-point GA-MS decoding has only a 0.1 dB gap compared to floating-point SP decoding under various scenarios of 5G NR standard specifications.
- A hardware-friendly optimized static schedule (OSS) is proposed to both improve error-correcting performance and reduce the worst-case decoding latency.
- We present a fully reconfigurable 5G NR LDPC decoder implementation using GA-MS decoding, compatible with all 5G NR LDPC codes. By adding data compression and approximation techniques, we achieve a significant reduction in memory overhead compared to explicit storage. The 28nm FD-SOI post-layout implementation has a core area of 1.823 mm^2, achieves a peak throughput of 24.42 Gbps at 895 MHz, and has an energy consumption of 12.56 pJ/bit with a supply voltage of 1.0 V.

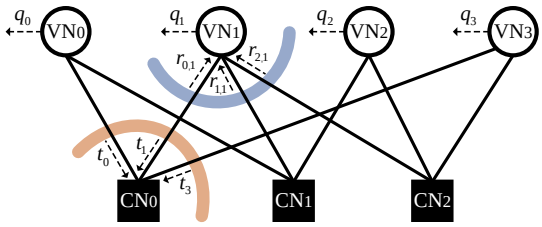


Fig. 1. A bipartite Tanner graph with VN update and CN update process.

The remainder of this paper is organized as follows: Section II provides symbol definitions and background on LDPC codes and decoding. Section III describes the proposed GA-MS decoding and its various optimizations. In Section IV, we present our 5G NR LDPC decoder architecture and the corresponding optimizations. Section V discusses implementation results. Finally, Section VI concludes the paper.

II. Preliminaries

Throughout this paper, we follow the definitions introduced below. Boldface small letters such as $\boldsymbol{u}$ denote vectors, where $\boldsymbol{u}[i]$ refers to the i-th element of $\boldsymbol{u}$. Boldface capital letters such as $\mathbf{B}$ represent matrices, where $\mathbf{B}[i][j]$ denotes the element at the i-th row of the j-th column of $\mathbf{B}$. Blackboard letters such as $\mathbb{S} = \{\cdot\}$ denote sets with $|\mathbb{S}|$ being the cardinality of $\mathbb{S}$. The hard decision function is defined as $\mathrm{HD}(x) = 1$ if $x < 0$ and $\mathrm{HD}(x) = 0$ if $x \geq 0$. The signum function, denoted as $\mathrm{sgn}(x)$, returns -1, 0, or 1, when x is negative, zero, or positive, respectively. For brevity, we use the term *floating-point* to specifically refer to double-precision floating-point.

$$\left\{\boldsymbol{x} \in \{0,1\}^N | \mathbf{H} \cdot \boldsymbol{x} = \mathbf{0}_{M\times 1}\right\}, \tag{1}$$

LDPC codes are linear block codes specified by a sparse $M \times N$ parity-check matrix (PCM) $\mathbf{H}$, as shown in (1), where $\boldsymbol{x}$ is a length-N binary column vector and $\mathbf{0}$ is a length-M all-zero vector. M denotes the number of parity checks and N denotes the code length. Furthermore, LDPC codes can also be described by a bipartite Tanner graph with a set of M CNs and N VNs [36]. If $\mathbf{H}[c][v] = 1$ for $0 \leq c < M$ and $0 \leq v < N$, the c-th CN is connected to the v-th VN on the Tanner graph. For each CN, we use $\mathbb{V}_c$ to represent the set of the adjacent VNs for the c-th CN and we use $\mathbb{C}_v$ to denote the set of the neighbours of the v-th VN. The number of neighbours that connect to a VN or a CN are referred to as their column- and row-degree, denoted as d_v and d_c, i.e., $|\mathbb{C}_v| = d_v$ and $|\mathbb{V}_c| = d_c$. Due to the quasi-cyclic property [37], [38], QC-LDPC codes are further described by a more structured $M_p \times N_p$ prototype matrix $\mathbf{H}_p$. Each entry of $\mathbf{H}_p$ can be expanded by substituting each element of $\mathbf{H}_p$ with a $Z \times Z$ identity matrix that is cyclically shifted by $\omega = \mathbf{H}_p[c][v] < Z$ for $0 \leq c < M_p$ and $0 \leq v < N_p$. The code parameter Z is referred to as the *lifting size*. If $\omega = -1$ or $\omega = 0$, the corresponding entry denotes a $Z \times Z$ all-zero or identity matrix, respectively.

A. 5G NR LDPC Codes

To enable rate-compatibility and incremental redundancy hybrid automatic repeat request (IR-HARQ), 5G NR adopts protograph-based raptor-like LDPC codes. Combined with the quasi-cyclic property, 5G NR LDPC codes can be derived from two base graphs (BG1 and BG2). Let K_u denote the number of information columns in the base graphs. In 5G standards, a complete BG1 comprises 46 rows and 68 columns (with the maximum $K_u = 22$), and a complete BG2 consists of 42 rows and 52 columns (with the maximum $K_u = 10$). A variety of code lengths and rates are attained by adjusting the lifting size Z and by puncturing the columns of the base graphs [6], [39],[1] which affects the number of parity-check bits. The leftmost two information columns in the base graphs are always punctured to boost transmission efficiency in practical scenarios. Let E and K denote the actual transmitted code length and information length after rate-matching. We can refer to 5G NR LDPC codes as (E, K) codes, where $K = K_u \cdot Z$ and where $R = \frac{K}{E}$ represents the code rate.

B. Layered LDPC Decoding

The error-correcting performance and hardware complexity of an LDPC decoder also depend on its decoding schedule. Two classical methods are flooding [8] and layered schedules [40], [41]. In contrast to the flooding schedule that updates all VNs together at the end of each iteration [8], layered decoding goes through the PCM and updates the connected VNs row by row. This update strategy results in faster convergence and a significant reduction in memory overhead, as fewer messages from VNs to CNs need to be stored.

Let q_v denote the *posterior* log-likelihood ratio (LLR) associated with the v-th VN, which is the aggregate value of all incoming messages and the channel LLR y_v. Let $r_{c,v}$ denote the message from the c-th CN to the v-th VN. We also define an intermediate variable t_v, corresponding to the message from the v-th VN at the current row. Layered decoding is completely defined by the aforementioned three message types: Q-message q_v, R-message $r_{c,v}$, and T-message t_v, as shown in Fig. 1. When processing the c-th row (i.e., the c-th CN) at the i-th iteration, layered decoding executes the updates shown in (2) (taking MS decoding as an example). Each of the three equations in (2) is only performed after the previous equation has been evaluated for all VNs, $0 \le v < N$.

$$t_v \leftarrow q_v - r_{c,v}, \tag{2a}$$

$$r_{c,v} \leftarrow \prod_{v' \in \mathbb{V}_c \setminus v} \operatorname{sgn}(t_{v'}) \cdot \min_{v' \in \mathbb{V}_c \setminus v} (|t_{v'}|), \tag{2b}$$

$$q_v \leftarrow t_v + r_{c,v}. \tag{2c}$$

First, the intermediate t_v values are computed on the fly using the stored q_v and $r_{c,v}$. Subsequently, the minimum and sign are selected, excluding the message along the current edge itself, to update all q_v and $r_{c,v}$ values. At the beginning of the first iteration, the q_v for $0 \le v < N$ are initialized by the channel LLRs y_v and the $r_{c,v}$ for $0 \le c < M$ and $0 \le v < N$ are set to zero. Once the maximum decoding iterations $I_{\max}$ are reached, the tentative codeword can be obtained by $\hat{x}_v = \mathrm{HD}(q_v), 0 \le v < N$. For 5G NR LDPC codes, the c-th row of $\mathbf{H}_p$ (i.e., BG1 and BG2) corresponds to the rows $c \cdot Z$ to $(c+1) \cdot Z - 1$ of $\mathbf{H}$. For simplicity, we call the rows $c \cdot Z$ to $(c+1) \cdot Z - 1$ of $\mathbf{H}$ the c-th layer.

[1]In 5G standards, the lifting size Z covers 51 distinct values ranging from 2 to 384 ($Z = i \times 2^n$, $i \in \{2, 3, 5, 7, 9, 11, 13, 15\}$, n are non-negative integers). For further details, please refer to Table 5.3.2-1 in [6].

C. Adjusted Min-Sum (A-MS) Decoding

In A-Min* decoding, the smallest magnitude of all incoming T-messages is identified first and two distinct outgoing magnitudes are computed using (3). Unlike MS decoding in (2), A-Min* decoding still requires complex SP calculations on all incoming T-messages after finding the minimum, which can result in a long decoding latency. However, based on the block-parallel architecture, using the same equations (3), A-MS decoding [21] can select the second smallest value when a new t_v arrives, and perform a box-plus operator with the new second minimum and the previous result to recursively update the outcome. This approach has the advantage that both results of (3) and the minimum can be obtained simultaneously after the arrival of the last valid block in the current row to avoid serial processing. The box-plus operator is approximated using simple LUTs to reduce hardware complexity. Corresponding to the edge with the minimum incoming magnitude, the outgoing R-message is referred to as a *critical message* [20] (consistent with the principle of SP decoding). For the remaining edges, the outgoing R-messages are called *non-critical messages* and are computed using all incoming messages (including the current edge itself). Hence, as a hardware-friendly decoding algorithm, A-MS decoding has a similar storage complexity as MS-based decoding and can achieve almost the same error-correcting performance as SP decoding.

$$r_{c,v} \leftarrow \begin{cases} \prod_{v' \in \mathbb{V}_c \setminus v} \operatorname{sgn}(t_{v'}) \cdot \boxplus_{v' \in \mathbb{V}_c \setminus v} |t_{v'}|, \text{ if } t_v \text{ is minimum}, \\ \prod_{v' \in \mathbb{V}_c \setminus v} \operatorname{sgn}(t_{v'}) \cdot \boxplus_{v \in \mathbb{V}_c} |t_v|, \text{ otherwise}. \end{cases} \tag{3}$$

Despite its superiority, conventional A-MS decoding still suffers from several points to be optimized. First, the influence of most incoming T-messages on the final outgoing R-messages is negligible based on our simulations, which means that most of the box-plus operators in (3) can be skipped to save computational complexity. In A-MS decoding, the identification of the minimum and the computation of the R-messages are tightly linked. This process relies on the block-parallel architecture that processes the incoming message block-by-block to resolve the data dependency, limiting its potential for increased parallelism. In addition, a straightforward way to design the box-plus operator in A-MS decoding is to approximate it as multiple serial small LUTs (as referred to [21]), leading to a relatively long data path due to the comparison, addition, and LUT operations.

III. Proposed High-Performance and Low-Complexity Decoding Algorithms

In this section, we propose a novel algorithm called GA-MS decoding, by extending the above A-MS decoding algorithm to a generalized form. Our algorithm offers high-performance and low-complexity decoding. By truncating the number of incoming messages in the CN processing, we can make a trade-off between the error-rate and computational complexity.

Moreover, combined with well-designed LUTs, quantization schemes, and approximation techniques, we provide a comprehensive performance analysis of fixed-point GA-MS decoding to demonstrate its stable and good error-rate across various code configurations and high-order modulations.

A. Generalized Adjusted Min-Sum (GA-MS) Decoding

Let t_a and t_b denote two arbitrary incoming T-messages. The complete box-plus operator between t_a and t_b is:

$$\begin{aligned} t_a \boxplus t_b &= 2\tanh^{-1}\left(\tanh\left(\frac{t_a}{2}\right)\cdot\tanh\left(\frac{t_b}{2}\right)\right) \\ &= \mathrm{sgn}(t_a)\cdot\mathrm{sgn}(t_b)\cdot\begin{pmatrix} \min\left(|t_a|,|t_b|\right) \\ +\ln\left(1+e^{-||t_a|+|t_b||}\right) \\ -\ln\left(1+e^{-||t_a|-|t_b||}\right)\end{pmatrix} \\ &\le \mathrm{sgn}(t_a)\cdot\mathrm{sgn}(t_b)\cdot\min\left(|t_a|,|t_b|\right), \end{aligned} \tag{4}$$

where the negative non-linear term $\triangle(t_a,t_b) := \ln\left(1+e^{-||t_a|+|t_b||}\right) - \ln\left(1+e^{-||t_a|-|t_b||}\right)$ is the reason why MS decoding always overestimates SP decoding [10], [11]. If $|t_a| \ll |t_b|$, we can further simplify (4) as

$$t_a \boxplus t_b \approx \mathrm{sgn}(t_a)\cdot\mathrm{sgn}(t_b)\cdot|t_a|, \tag{5}$$

which means that the smallest incoming magnitude dominates in (4). Hence, in CN processing, more emphasis is placed on these incoming T-messages with smaller magnitudes, assigning them greater weights in the outgoing numerical calculation. Similar to [20], we define a new set called $\mathbb{V}_c^\gamma$ to only contain the first γ smallest magnitude values. Note that if $d_c \ge \gamma$, $|\mathbb{V}_c^\gamma| = \gamma$ holds, but if $d_c < \gamma$, $\mathbb{V}_c^\gamma$ is equivalent to $\mathbb{V}_c$ without any message truncation (i.e., $|\mathbb{V}_c^\gamma| = |\mathbb{V}_c| = d_c$). The corresponding update of (3) is as follows, where we can flexibly configure the parameter γ to adjust the number of incoming messages used in CN processing.

$$r_{c,v} \leftarrow \begin{cases} \prod\limits_{v'\in\mathbb{V}_c\setminus v}\mathrm{sgn}(t_{v'})\cdot\boxplus\limits_{\tilde{v}'\in\mathbb{V}_c^\gamma\setminus v}|t_{\tilde{v}'}|, \text{ if } t_v \text{ is minimum,} \\ \prod\limits_{v'\in\mathbb{V}_c\setminus v}\mathrm{sgn}(t_{v'})\cdot\boxplus\limits_{\tilde{v}\in\mathbb{V}_c^\gamma}|t_{\tilde{v}}|, \text{ otherwise.}\end{cases} \tag{6}$$

As shown in (7), $|t_{v^*}|$ is the $(\gamma+1)$-th smallest incoming magnitude. It is intuitive to prove that when γ is larger, the outgoing $r_{c,v}$ approaches the original A-Min* result.

$$\boxplus\limits_{\tilde{v}\in\mathbb{V}_c^\gamma}|t_{\tilde{v}}| \ge \left(\boxplus\limits_{\tilde{v}\in\mathbb{V}_c^\gamma}|t_{\tilde{v}}|\right)\boxplus|t_{v^*}| = \boxplus\limits_{\tilde{v}\in\mathbb{V}_c^{\gamma+1}}|t_{\tilde{v}}| \ge \ldots \ge \boxplus\limits_{v\in\mathbb{V}_c}|t_v|. \tag{7}$$

However, if the value of γ is relatively small, GA-MS decoding truncates too much information, resulting in an obvious performance loss. In order to compensate for this degradation, we introduce an additional offset β in (8) to reasonably scale the outgoing R-message and effectively alleviate the overestimation phenomenon as

$$r_{c,v} \leftarrow \begin{cases} \prod\limits_{v'\in\mathbb{V}_c\setminus v}\mathrm{sgn}(t_{v'})\cdot\max\left(\left(\boxplus\limits_{\tilde{v}'\in\mathbb{V}_c^\gamma\setminus v}|t_{\tilde{v}'}|\right)-\beta,0\right), \\ \text{if } t_v \text{ is minimum,} \\ \prod\limits_{v'\in\mathbb{V}_c\setminus v}\mathrm{sgn}(t_{v'})\cdot\max\left(\left(\boxplus\limits_{\tilde{v}\in\mathbb{V}_c^\gamma}|t_{\tilde{v}}|\right)-\beta,0\right), \\ \text{otherwise.}\end{cases} \tag{8}$$

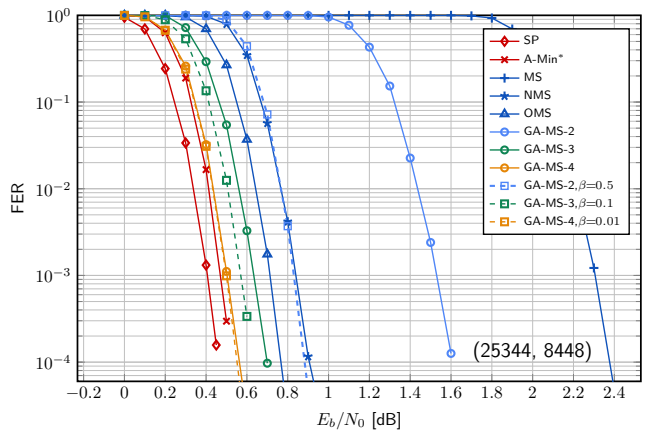


Fig. 2. Floating-point FER comparison of SP, A-Min*, MS, NMS, OMS, and GA-MS decoding with β and $\gamma \in \{2,3,4\}$ for 5G NR LDPC codes (BG1, $R=\frac{1}{3}$, $Z=384$, and $K_u=22$) using QPSK and $I_{\max}=15$.

It is worth noting that, compared to the original A-MS decoding algorithm [21], the proposed GA-MS decoding has a completely different decoding process, which first collects the γ smallest incoming magnitudes just as the MS-based decoders do and then executes $\gamma-1$ approximate box-plus operators and an additional subtraction together in the CN processing. The above improvement endows GA-MS decoding with a similar efficient hardware architecture as classical MS-based decoders [24], [25], which significantly facilitates the corresponding decoder implementation (discussed in Section IV).

The floating-point FER performance comparison of SP decoding and our GA-MS decoding with varying values of γ and β is provided in Fig. 2. The simulation is conducted with 5G NR LDPC codes (BG1, $R=\frac{1}{3}$, $Z=384$, $K_u=22$) with quadrature phase shift keying (QPSK) over an additive white Gaussian noise (AWGN) channel and $I_{\max}=15$. To fully demonstrate the capabilities of OMS and NMS decoding [12], we finely adjust the offset and the normalization factor for their optimal performance. Specifically, for each E_b/N_0 point, we sweep offsets of OMS within the range of 0.3 to 0.7, in increments of 0.05, and normalization factors of NMS between $\frac{1}{2}$ and $\frac{15}{16}$, in $\frac{1}{16}$ increments to plot the FER for the best values. This approach ensures a fair comparison with GA-MS decoding. In all following captions referring to GA-MS decoding, the last digit of the label (e.g., GA-MS-2) denotes the number γ of used minima during the decoding. When $\gamma=2$, GA-MS decoding is simplified to a near-MS algorithm (GA-MS-2) that surpasses MS decoding by 0.75 dB, but still underperforms OMS and NMS. If we increase γ to 3 or 4, GA-MS decoding enables a significant improvement, closing the gap with SP decoding to 0.25 dB and 0.1 dB at FER $=10^{-3}$, respectively. With respect to determining the compensation factor, a moderate subtraction of β can cause a notable performance improvement when $\gamma \in \{2,3\}$. For example, GA-MS-3 decoding with a value of β equal to 0.1 only has a gap compared to SP decoding by 0.16 dB and outperforms OMS decoding by 0.15 dB at FER $=10^{-3}$. However, as γ increases, especially for $\gamma \ge 4$, the original outgoing R-message in (6) is accurate enough to approach the original A-Min*result so that the impact of β is rapidly reduced. As shown in Fig. 2, GA-MS-4 decoding without

TABLE I
COMPARISON OF COMPUTATIONAL COMPLEXITY INCURRED BY VARIOUS LDPC DECODING ALGORITHMS.

Algorithms	SP† [19]	A-Min*† [19]	MS [11]	OMS [12]	NMS‡ [12]	A-MS [21]	**GA-MS**
Comparisons	-	$(d_c-1)\cdot M$	$(2d_c-3)\cdot M$	$(2d_c-3)\cdot M$	$(2d_c-3)\cdot M$	$(2d_c-3)\cdot M$	$\boldsymbol{(\gamma\cdot d_c-\frac{(\gamma+1)\cdot\gamma}{2})\cdot M}$
Additions	$d_v\cdot N+(2d_c-1)\cdot M$	$d_v\cdot N+(2d_c-1)\cdot M$	$d_v\cdot N$	$d_v\cdot N+2M$	$d_v\cdot N+2M$	$d_v\cdot N+(3d_c-6)\cdot M$	$\boldsymbol{d_v\cdot N+2M}$
LUTs	$2d_c\cdot M$	$(d_c-1)\cdot M$	-	-	-	$(2d_c-4)\cdot M$	$\boldsymbol{(\gamma-1)\cdot M}$
Memory	$d_c\cdot M+N$	$3M+N$	$2M+N$	$2M+N$	$2M+N$	$3M+N$	$\boldsymbol{2M+N}$

† The hyperbolic-tangent functions in CN processing for SP, A-Min*, and A-MS are approximated by LUTs in [19], [21], which refer to Table I of [18].
‡ The multiplication in NMS decoding is implemented by shift-and-add operations to reduce the computational complexity, similarly to [34].

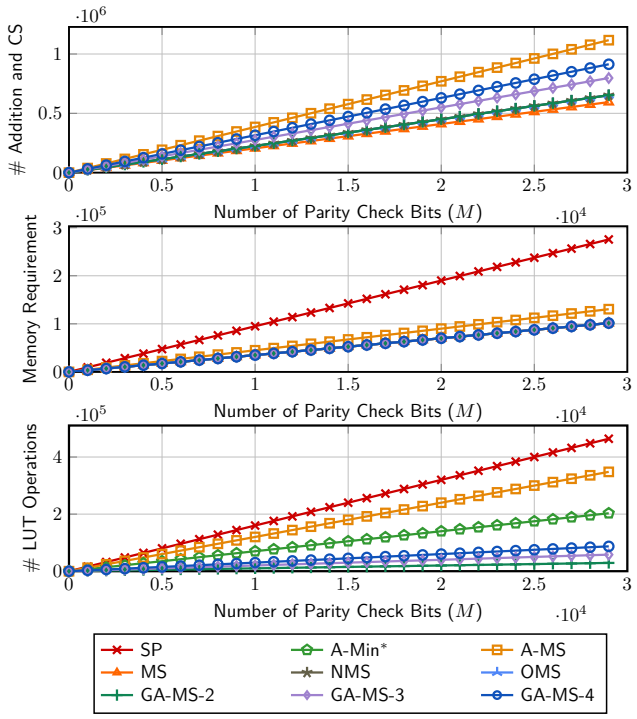


Fig. 3. Comparison of computational complexity (addition and CS, memory requirement, and LUT operations) among GA-MS decoding and various existing LDPC decoding algorithms.

β has nearly the same error-correcting performance as that with a compensation factor $\beta = 0.01$. Besides, GA-MS-4 also exhibits almost the same performance as the original A-Min* decoding, which shows that $\gamma = 4$ is precise enough for the result of A-Min* decoding in CN processing.

Table I presents a summary of our analysis on the computational complexity per iteration for the proposed GA-MS decoding algorithm, in comparison with various other LDPC decoding algorithms [11], [12], [19], [21], [34]. The analysis reveals that the number of comparisons in GA-MS decoding increases with γ due to the internal sorting corresponding to multiple minima. Nevertheless, only a single addition operation occurs outside the entire box-plus operator (as shown in (8)) within each CN processing unit. As a result, GA-MS decoding outperforms the benchmark A-MS decoding in terms of both the number of addition and compare-select (CS) operations and the number of LUTs, especially when $\gamma \in \{2, 3, 4\}$. GA-MS decoding also has the same memory consumption as MS-based decoding [11], [12], which involves storing N channel LLRs and two outgoing messages corresponding to each CN.

To further demonstrate the advantages of GA-MS decoding, we provide comparative plots in Fig. 3. These plots chart a function of the parity-check length M (represented on the horizontal axis) for regular LDPC codes ($R = \frac{1}{3}$, $d_c = 8$, $d_v = 5$), to be consistent with the example in [35]. Considering the maximum code length of 5G NR LDPC codes, specified for $M = 17664$, GA-MS-3 decoding exhibits a reduction of 28.7% addition and CS operations and 22.3% memory overhead reduction compared to A-MS decoding. It is noteworthy that GA-MS decoding requires slightly more LUT operations relative to OMS and NMS decoding. Despite this, based on the message truncation in the CN processing, GA-MS-3 decoding reduces 87.5% and 83.3% of LUT operations compared to SP and A-MS decoding, respectively.

In conclusion, the proposed GA-MS decoding offers several distinct advantages over previous works. First, by adjusting the parameter γ to truncate the number of incoming messages, GA-MS decoding achieves a good trade-off between error-correcting performance and computational complexity. Second, even with an increment of γ, GA-MS decoding still only computes two distinct outgoing messages using (8), with no additional memory overhead. In terms of hardware implementation, similar to an MS-based architecture, GA-MS decoding can completely decouple the procedures of the minima collection and approximate box-plus operators to maintain a high operating frequency, which is further discussed in Section IV.

B. Quantization of GA-MS Decoding

In this section, we focus on quantization techniques to enhance the fixed-point performance of GA-MS decoding. We adopt a uniform quantization:

$$\mathbf{\Lambda}(y_v) = \mathrm{sgn}(y_v)\cdot\min\left(\left\lfloor\frac{|y_v|}{\delta}+0.5\right\rfloor, 2^{B-1}-1\right), \quad (9)$$

where the bit-width B is composed of one sign bit, B_i integer bits, and B_f fractional bits. The channel gain factor δ is defined as $1/2^{B_\mathrm{f}}$ to scale the inputs. During the LDPC decoding process, all propagating messages can be categorized into two types: q_v and t_v (associated with VNs), and $r_{c,v}$ (associated with CNs). As $r_{c,v}$ is only based on the minimum of incoming T-messages, unlike an aggregate LLR, it has a smaller dynamic range. Namely, $r_{c,v}$ can utilize fewer quantization bits than q_v and t_v. Therefore, we adopt a quantization scheme denoted as $(B_\mathrm{VN}, B_\mathrm{CN}, B_\mathrm{f})$, where B_VN and B_CN are the numbers of quantization bits for messages associated with VNs and CNs, respectively and all messages have B_f fractional bits.

1) Decoding Algorithm: Quantized GA-MS-γ decoding (for layered decoding of QC-LDPC codes) is outlined in Algorithm 1. As mentioned in Section II-B, our GA-MS decoding requires memory for three message types ($\boldsymbol{q}_v[k]$, $\boldsymbol{t}_v[k]$,

Algorithm 1: Quantized GA-MS-γ Decoding (QC-Layered Version)

Initialize: $\boldsymbol{q}_v \leftarrow \boldsymbol{y}_v$, $\boldsymbol{r}_{c,v} \leftarrow \mathbf{0}_{Z\times 1}, \forall c, v$

// Iterative decoding
1 **for** $i = 0$ **to** $I_{\max} - 1$ **do**
2 **for** $c = 0$ **to** $M_p - 1$ **do**
// Initialize γ minima and sign bit
3 $\mathbb{M} = [\boldsymbol{m}_1, \boldsymbol{m}_2, \ldots, \boldsymbol{m}_\gamma] \leftarrow \infty \cdot [\mathbf{I}_{Z\times 1}, \mathbf{I}_{Z\times 1}, \ldots, \mathbf{I}_{Z\times 1}]$
4 $\boldsymbol{s} \leftarrow \mathbf{I}_{Z\times 1}$
// Phase 1: MIN
5 **for** $v \in \mathbb{V}_c$ **do**
6 $\omega_v = \mathbf{H}_p[c][v]$, $\boldsymbol{t}_v \leftarrow$cyclicShift$(\boldsymbol{q}_v, \omega_v) - \boldsymbol{r}_{c,v}$
// Sort γ minima, find index of $\boldsymbol{m}_1$
7 $[\mathbb{M}, \boldsymbol{v}_{\min}] \leftarrow$sortMin$(\mathbb{M}, |\boldsymbol{t}_v|)$
8 $\boldsymbol{s} \leftarrow \boldsymbol{s} \cdot \mathrm{sgn}(\boldsymbol{t}_v)$
// Phase 2: SEL
9 **for** $v \in \mathbb{V}_c$ **do**
// LUT-based approx using γ minima
10 $\boldsymbol{m}_{\mathrm{LUT}} \leftarrow$LUTMin$(\mathbb{M}, \boldsymbol{v}_{\min}, v)$
// update R- and Q-messages
11 $\boldsymbol{r}_{c,v} \leftarrow \boldsymbol{s} \cdot \mathrm{sgn}(\boldsymbol{t}_v) \cdot \boldsymbol{m}_{\mathrm{LUT}}$
12 $\boldsymbol{q}_v \leftarrow$cyclicShift$(\boldsymbol{t}_v + \boldsymbol{r}_{c,v}, Z - \omega_v)$

Return : $\hat{\boldsymbol{x}}_v = \mathrm{HD}(\boldsymbol{q}_v), \forall v$

Algorithm 2: LUTMin()

1 **for** $k = 0$ **to** $Z - 1$ **do**
2 **if** $v \neq \boldsymbol{v}_{\min}[k]$ **then**
3 $\boldsymbol{m}_{\mathrm{LUT}}[k] = \boldsymbol{m}_1[k]$ // non-critical messages
4 **for** $t = 2$ **to** γ **do**
5 $\boldsymbol{m}_{\mathrm{LUT}}[k] = \mathrm{LUT}(\boldsymbol{m}_{\mathrm{LUT}}[k], \boldsymbol{m}_t[k])$
6 **else**
7 $\boldsymbol{m}_{\mathrm{LUT}}[k] = \boldsymbol{m}_2[k]$ // critical message
8 **for** $t = 3$ **to** γ **do**
9 $\boldsymbol{m}_{\mathrm{LUT}}[k] = \mathrm{LUT}(\boldsymbol{m}_{\mathrm{LUT}}[k], \boldsymbol{m}_t[k])$

$\boldsymbol{r}_{c,v}[k]$), which are all Z-dimensional vectors for $0 \leq v < N_p$, $0 \leq c < M_p$, and $0 \leq k < Z$. Similar to [24], we divide the entire algorithm (excluding early termination) into two primary phases (MIN and SEL) to decouple the computation of (2) into several procedures that can be executed separately in different clock cycles. This division is beneficial for hardware implementation to improve the maximum operating frequency. First, we initialize the vectors $\boldsymbol{q}_v$ using the input channel LLR vectors $\boldsymbol{y}_v$ and set the vectors $\boldsymbol{r}_{c,v}$ to all-zero vectors. The algorithm proceeds in a layer-wise manner (for each row of $\mathbf{H}_p$), and the message computation is executed iteratively only for the columns $v \in \mathbb{V}_c$ of $\mathbf{H}_p$ for which $\mathbf{H}_p[c][v] \neq -1$. Note that, in line 7, the magnitude set $\mathbb{M}$ only contains the magnitude information of collected minima, and we need an extra vector $\boldsymbol{s}$ to store all sign bits.

The first phase, referred to as MIN, calculates the intermediate vector $\boldsymbol{t}_v$ and gathers γ minima vectors for the $\boldsymbol{t}_v$ in the current layer. As a new vector $\boldsymbol{t}_v$ arises, the sortMin() function updates and maintains an ascending ordered set of γ minima vectors, denoted as $\mathbb{M} = \{\boldsymbol{m}_1, \boldsymbol{m}_2, \ldots, \boldsymbol{m}_\gamma\}$, where $\boldsymbol{m}_1$ (a length-Z vector) contains the first minima of Z rows, $\boldsymbol{m}_2$ holds the second minima of Z rows, and so on. In addition, the sortMin() function also keeps track of the column index vector $\boldsymbol{v}_{\min}$ of $\boldsymbol{m}_1$ to distinguish the calculation of critical and non-critical messages in (6).

The second phase, referred to as SEL, is activated after the MIN phase has swept across the entire layer. During the SEL phase, the vectors $\boldsymbol{r}_{c,v}$ and $\boldsymbol{q}_v$ of the current iteration are block-wise updated based on the collected vectors $\mathbb{M}$, $\boldsymbol{v}_{\min}$, and $\boldsymbol{s}$. The LUTMin() function shown in Algorithm 2 corresponds to (6) in Section III-A. This function represents the recursive processing that repeatedly invokes the box-plus LUT at lines 5 and 9 (the efficient LUT design is discussed below), and the only difference is the removal of the vector $\boldsymbol{m}_1$ from the calculation if the current edge with the minimum incoming message. As the new vector $\boldsymbol{m}_{\mathrm{LUT}}$ only contains the magnitude information, the updated vector $\boldsymbol{r}_{c,v}$ is computed by using both the sign vector $\boldsymbol{s}$ and the vector $\boldsymbol{m}_{\mathrm{LUT}}$. Finally, the updated vector $\boldsymbol{q}_v$ is cyclically shifted

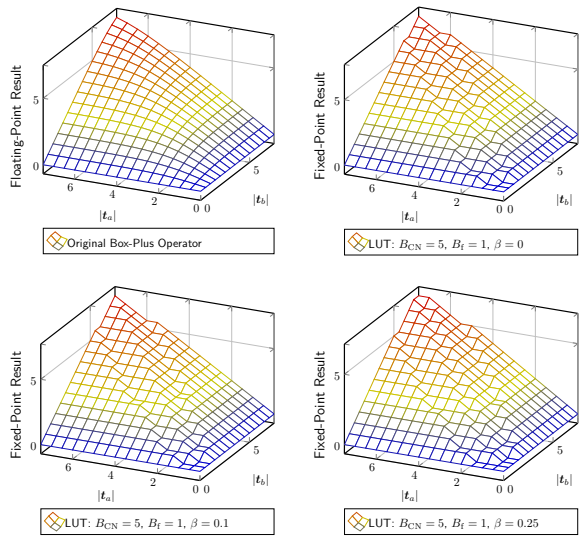


Fig. 4. Comparison of the results between the original box-plus operator and LUTs with varying values of β in (11).

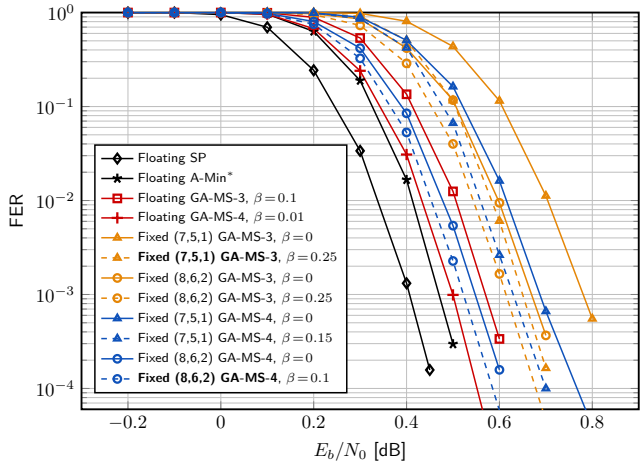


Fig. 5. Fixed-point FER comparison of GA-MS decoding with different quantization strategies and varying values of β for 5G NR LDPC code (BG1, $R = \frac{1}{3}$, $Z = 384$, and $K_u = 22$) using QPSK and $I_{\max} = 15$.

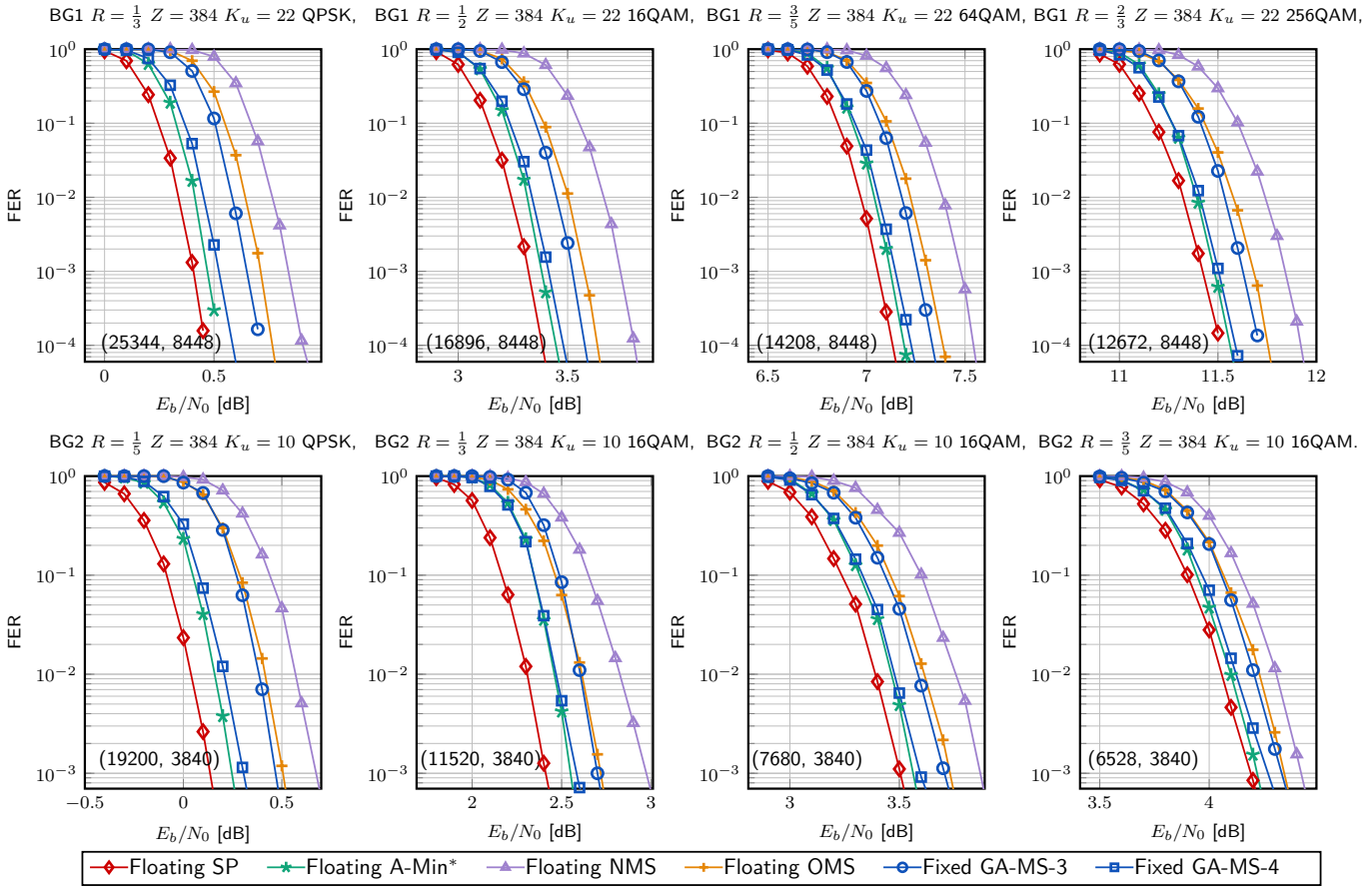


Fig. 6. Fixed-point FER of GA-MS decoding for various 5G NR LDPC codes and high-order modulations over an AWGN channel, where $I_{\max}=15$ and the parameters of OMS and NMS decoding have been carefully tuned.

using the inverse rotation $Z-w_v$. Note that this additional rotation for Q-messages can be eliminated in hardware [25], as discussed in Section IV-A.

2) LUT Design: The box-plus LUT design is critical in determining error-correcting performance and computational complexity of GA-MS decoding. The truncation of incoming messages to γ minima, which are dominant in CN processing, brings advantages for GA-MS decoding, as it helps to design LUTs to approximate the original result.

If t_a and t_b have been quantized using the $(B_{\text{VN}}, B_{\text{CN}}, B_{\text{f}})$ scheme, the non-linear term in (4) can be quantized as

$$\mathbf{\Lambda}(\triangle(t_a,t_b)) = \text{sgn}\left(\triangle(t_a,t_b)\right)\cdot\left\lfloor\frac{|\triangle(t_a,t_b)|}{\delta}+0.5\right\rfloor. \quad (10)$$

To realize a similar effect as the subtraction of a compensation factor in (8), we move the β into (10) to introduce (11) as the approximation of the entire box-plus operator,

$$\text{LUT}(t_a,t_b) = \text{sgn}(t_a)\cdot\text{sgn}(t_b)\cdot \left(\max\left(\min(|t_a|,|t_b|) - \left\lfloor\frac{|\triangle(t_a,t_b)|}{\delta}+\beta+0.5\right\rfloor, 0\right)\right). \quad (11)$$

Fig. 4 demonstrates the impact of β in (11) on the LUT results. As β gets larger, the difference between the LUT outcome and the original box-plus operator grows. However, for two fully distinct magnitudes, the LUT result directly equals the minimum, corresponding to (5). In general, if γ is small, a larger β is needed in (11) to compensate for performance degradation caused by truncation. The parameter β extends the design space of the LUTs for our GA-MS decoding, facilitating the exploration of various quantization strategies and high-order modulations. Based on a well-designed quantization strategy and β in (11), we can generate an efficient LUT and use (6) as our fixed-point GA-MS decoding to approach the floating-point performance of (8).

3) Comparison of Different Quantization Strategies: Fig. 5 presents the fixed-pointed FER performance of GA-MS decoding under various quantization strategies and with different β values in (11) (using the same code configuration as Fig. 2). First, we adopt two popular quantization schemes, $(7,5,1)$ and $(8,6,2)$, from the 5G NR LDPC decoders in [15], [30], [32], [34], [35], to achieve a balance between high performance and hardware (including memory) complexity. Second, we provide a selection of empirical β values in (11) to improve quantized GA-MS decoding, which can be applied in the subsequent analysis and implementations.

Under the quantization schemes $(7,5,1)$ and $(8,6,2)$, fixed-point GA-MS-3 decoding with $\beta=0$ has a loss of 0.2 dB and 0.1 dB from floating-point GA-MS-3. When using $\beta=0.25$ for $(7,5,1)$, the performance gap compared to floating-point GA-MS-3 reduces to 0.08 dB. However, GA-MS-4 decoding demands greater precision for message propagation. The high-resolution $(8,6,2)$ scheme thus aligns well with GA-MS-4 decoding to enable more precise numerical calculation based on minima. In Fig. 5, the fixed-point performance using $(8,6,2)$ with $\beta=0.1$ almost has the same performance as

that of floating-point GA-MS-4, which exhibits only a 0.1 dB gap relative to floating-point SP decoding. Consequently, for the following sections, our fixed-point GA-MS-3 decoding employs the quantization scheme $(7, 5, 1)$, while the fixed-point GA-MS-4 algorithm utilizes the quantization scheme $(8, 6, 2)$.

C. Comprehensive Performance Analysis

To evaluate the error-correcting capability of the proposed GA-MS decoding in practical scenarios, we conduct simulations on various 5G NR LDPC codes and with different high-order modulations [42], comparing them with other classical LDPC decoding algorithms [8], [12], [19] for detailed performance analysis. For all modulations, channel LLRs y_v are obtained using the max-log-MAP method over an AWGN channel. Notably, the configurable β in (11) demonstrates robustness across a wide range of code rates and modulations, despite being relatively sensitive to the base graph selection. Hence, we present empirical optimal values of β in (11) for BG1 and BG2 in Fig. 6, respectively.

For BG1, we set $\beta = 0.25$ and $\beta = 0.1$ for GA-MS-3 and GA-MS-4 decoding, respectively. Numerical results show that fixed-point GA-MS-3 decoding can approach the performance of SP decoding within 0.25 dB before FER $= 10^{-3}$, while fixed-point GA-MS-4 decoding exhibits a gap of 0.1 dB compared to SP decoding. This gap gradually diminishes as the code rate increases. For BG2, we adopt a β value of 0.1 in (11) for both GA-MS-3 and GA-MS-4 decoding. With $R = \frac{1}{5}$ with QPSK, fixed-point GA-MS-4 decoding has an improvement of around 0.19 dB compared to floating-point OMS decoding. Besides, at medium to high code rates, our fixed-point GA-MS-4 decoding performs almost the same as floating-point A-Min* decoding on both BG1 and BG2.

IV. Proposed 5G LDPC Decoder Architecture

In this section, we present the implementation of a fully reconfigurable 5G NR LDPC decoder, incorporating our GA-MS decoding and all the aforementioned algorithmic optimizations. This decoder is compatible with all 5G NR LDPC codes. We provide a comprehensive description of each core component, which contains three embedded memory banks (referred to as the Q-memory, the T-memory, and the R-memory), a pool of node computation units (NCUs), a cyclic shifter unit (CSU), and a controller to complete layered GA-MS decoding algorithm in a block-parallel iteratively decomposed fashion.

A. High-Level Overview

Fig. 7 illustrates the high-level architecture of our fully reconfigurable 5G NR LDPC decoder, which builds upon the baseline architecture proposed in [24], [25]. To enable layered GA-MS decoding in a block-parallel fashion, we decompose the processing of each layer into multiple cycles. In each cycle, we update all Z parity checks for the current block simultaneously (i.e., instantiating $Z_{\max}$ processing units for 5G) and further optimize the processing units by decoupling them into the MIN and SEL units to enhance the operating frequency. As shown in Fig. 7, the MIN units perform several tasks in each cycle. They read the corresponding vectors $\boldsymbol{q}_v$ and $\boldsymbol{r}_{c,v}$ from the associated Q- and R-memories, compute the intermediate vector $\boldsymbol{t}_v$, and write the results to the associated

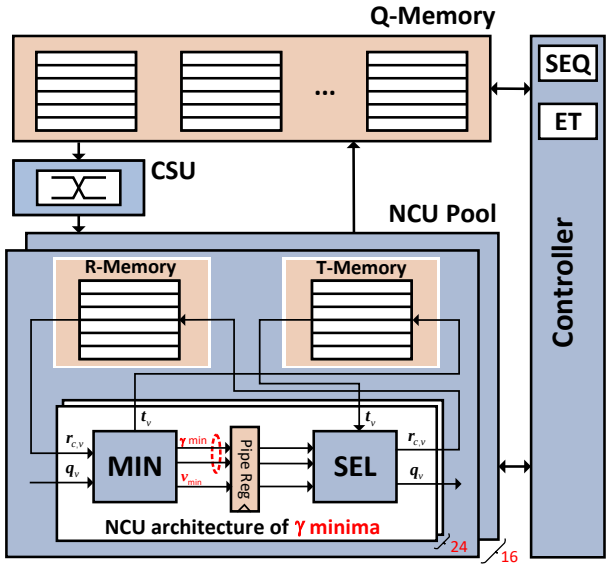


Fig. 7. High-level architecture of fully reconfigurable 5G NR LDPC decoder based on GA-MS decoding.

T-memory. Additionally, the MIN units also track the set $\mathbb{M}$ with γ minima vectors in Algorithm 1 and update pipeline registers of the NCUs at the end of each layer. Meanwhile, based on the previously stored $\mathbb{M}$, the SEL units read the latest vector $\boldsymbol{t}_v$ from the associated T-memory and update the vectors $\boldsymbol{q}_v$ and $\boldsymbol{r}_{c,v}$. It is noteworthy that the MIN and SEL units are pipelined to process two consecutive layers (the MIN units always work ahead of the SEL units). Moreover, to rotate the Q-messages according to the QC-LDPC prototype matrix, we implement a CSU to perform a cyclic left-shift by $w = \mathbf{H}_p[c][v]$ for the read vector $\boldsymbol{q}_v$. Instead of re-rotating the updated Q-messages when writing them back to the Q-memory, the rotation value of the Q-messages is tracked during the processing to avoid a second CSU instantiation [25]. Namely, in the hardware implementation, we remove the cyclicShift() function of line 12 and change line 6 to (12) in Algorithm 1

$$\omega_v = \mathrm{mod}\ (Z + \mathbf{H}_p[c][v] - \mathbf{H}_p[c-1 \oplus M_p][v],\ Z), \quad (12)$$

where the term $\mathbf{H}_p[c-1 \oplus M_p][v]$ records the rotation value of the current block at the previous layer.

In our 5G NR LDPC decoder, an early termination technique (same as [24], [25]) called partial parity checks (PPCs) is adopted to terminate converged codewords and improve average throughput. In the SEL units, each row of the prototype matrix yields a set of Z parity checks which are combined into a single PPC. If all PPCs are correct, the decoding procedure terminates prematurely after processing all M_p rows of $\mathbf{H}_p$. While the PPC approach is sub-optimal in terms of the number of decoding iterations compared to the conventional complete $\mathbf{H} \cdot \hat{\boldsymbol{x}} = 0$, it can be done efficiently in a block-parallel decoder.

Finally, the whole decoding process is orchestrated by the controller, which reads from a list of instructions in the SEQ memory and coordinates the components of our 5G NR LDPC decoder. Notably, this controller can be configurable by the lifting size Z (i.e., sub-block size), the prototype matrix $\mathbf{H}_p$, and $I_{\max}$, which enables decoder reconfigurability.

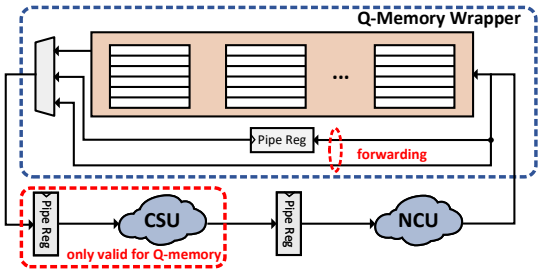


Fig. 8. Memory wrapper for the Q-memories, which has the same structure as that of the T-memories.

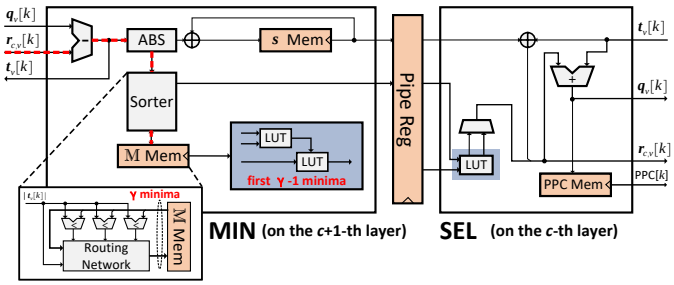


Fig. 9. Architecture of the k-th NCU in the NCU pool for our GA-MS decoding, which illustrates the critical path (red dotted line) and internal sorter.

TABLE II
MEMORY SIZES OF OUR 5G NR LDPC DECODER†.

Memory	(7,5,1) Q	T	R-sign	R-mag	(8,6,2) Q	T	R-sign	R-mag
Width [bit]	168	168	24	312	192	192	24	360
Depth [word]	68	68	316	46	68	68	316	46
Instance Count	16	16	16	16	16	16	16	16
Capacity‡ [KB]	22.31	22.31	14.81	28.03	25.5	25.5	14.81	32.34

† The number of all the above memory instances is 16.
‡ Memory capacities are the sum of 16 instances of each memory.

B. Decoder Memories

In this section, we introduce decoder memories with grouping and data compression techniques to fit any 5G NR LDPC codes into the allocated memories.

1) Grouping: As mentioned in Section II-A, 5G NR LDPC codes feature 51 distinct lifting sizes. When Z is less than $Z_{\max}$, it is inefficient and energy-consuming to consistently operate the decoder at maximum parallelism $Z_{\max}$. Therefore, our LDPC decoder, including the NCU pool, datapaths, and memories, demands a fine-grained structure.

As illustrated in Fig. 7, we pack 24 NCUs into a single group and thus divide the whole $Z_{\max} = 384$ NCUs into 16 groups that are driven by different gated clocks. Each group shares a collection of independent memories to maintain macros with reasonable sizes and avoid extremely small (i.e., inefficient) macros.

Fig. 8 depicts the corresponding memory wrapper for the Q-memories. Each Q memory features a width of $24B_{\text{VN}}$ bits and a depth of 68 words, with 24 denoting the number of Q-messages within each word, and 68 corresponding to N_p in BG1. All the Q-memories share the same read and write addresses and thus merge a complete vector of length-$Z_{\max}$ messages to be read and written simultaneously. As the same Q-memory block may undergo multiple updates during each iteration, we implement two forwarding paths to prevent possible memory conflicts and enhance throughput. Notably, the depth of the T-memories can theoretically be reduced from 68 to 19 words, due to $d_c^{\max} = 19$ in 5G base graphs. However, this reduction necessitates a complicated peripheral circuit for address mapping [15]. Furthermore, the memory area reduction for such small memories is less than proportional to the reduction in the number of words. As a result, the T-memories in our decoder still maintain the same depth as the Q-memories to offer simpler control logic.

2) Compressed Format of R-Messages: In this decoder, we employ a compressed data format for R-messages instead of using explicit storage and the clipping of Q-messages like in [24], [25]. In (6), the outgoing R-message has only two distinct magnitudes in each row (i.e., critical and non-critical messages). By excluding all sign bits, we can store a compressed word (only comprising two magnitudes and the column index of the critical message) to recover all R-messages for each row. Hence, each group of the R-memories consists of two parts: R-sign and R-mag memories. As mentioned in Section III-B, all sign bits of the R-messages demand explicit storage, the width of each R-sign memory is 24 bits and the depth is 316 words (i.e., maximum number of non-zero entries in BG1). It is noticeable that the column index value can be compressed to require only 5 bits (instead of the 7 bits required to store the full column index), due to $d_c^{\max} = 19$ in 5G. In the decoder, we implement a LUT to perform this index compression operation. For each R-mag memory, the width is $24 \times (2 \times (B_{\text{CN}} - 1) + 5)$ bits and the depth is 46 words (corresponding to M_p in BG1). Compared to conventional explicit storage, the above compressed technique can save approximately 42.2% and 46.9% of bits for the R-memories with the $(7, 5, 1)$ and $(8, 6, 2)$ quantization schemes, respectively.

In our 5G NR LDPC decoder, the detailed memory configurations are outlined in Table II, with each memory instantiated 16 times. Hence, the total memory capacities for the two quantization schemes are 87.47 KB for $(7, 5, 1)$ and 98.16 KB for $(8, 6, 2)$.

C. Node Computation Units (NCUs)

The architecture of the k-th NCU ($0 \le k < Z_{\max}$) in the NCU pool is illustrated in Fig. 9. Internal pipeline registers separate the NCU computation into two phases. The MIN unit iteratively computes the intermediate message $\boldsymbol{t}_v[k]$ and collects the updated γ minima at the $((c+1) \cdot Z + k)$-th row, while the SEL unit concurrently updates the corresponding $\boldsymbol{q}_v[k]$ and $\boldsymbol{r}_{c,v}[k]$ at the $(c \cdot Z + k)$-th row. Compared to the original NCU for layered OMS decoding [24], [25] (only a simple subtraction with a fixed offset in the SEL unit), our GA-MS decoding in (6) needs a set of LUTs, as shown in Algorithm 2. This additional computation introduces latency in the SEL unit which degrades the maximum operating frequency of the decoder. To alleviate this issue, we further decouple the partial calculation of (6) and the updating of $\boldsymbol{q}_v[k]$ and $\boldsymbol{r}_{c,v}[k]$ into different cycles to balance the datapaths.

For instance, when processing the non-critical message of (6), we need to sequentially invoke $\gamma - 1$ LUTs for γ minima inputs to calculate the result. Before the MIN unit reaches the

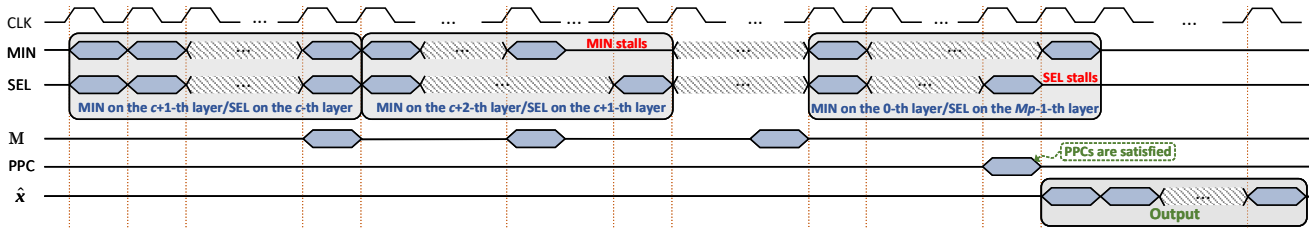


Fig. 10. Example timing schedule of the proposed 5G NR LDPC decoder based on GA-MS decoding.

last block of each row, the iteratively updated $\mathbb{M}$ memory has already gathered (at least) $\gamma - 1$ correct minima. Hence, we can move the calculation of $\gamma - 2$ LUTs, based on the first $\gamma - 1$ minima of the $\mathbb{M}$ memory, to the MIN unit in advance. Note that this result is only intermediate due to the absence of one minimum. Upon arriving at the end of each row, the MIN unit forwards this intermediate result and γ minima to pipeline registers. The SEL unit only needs to perform one LUT based on the fully updated γ minima to accurately compute the non-critical message. This approach can significantly optimize the datapaths without any stalls. The critical message of (6) is processed similarly. Moreover, due to the existing strict order of γ minima (in the $\mathbb{M}$ memory), we can implement a pruned $\gamma+1 \rightarrow \gamma$ sorter to eliminate the redundant comparators in the MIN unit. This sorter, which comprises a routing network and a layer of γ comparators as shown in Fig. 9, can be considered as a special case of low-latency *rank-order sorters* [43].

D. Timing Schedule and Latency Analysis

Fig. 10 demonstrates the timing schedule of our 5G NR LDPC decoder from the perspective of the NCUs. As discussed before, the MIN and SEL units are pipelined to work on two consecutive layers to balance the datapaths. However, this approach inevitably introduces stalls in the LDPC decoder. In general, these stalls are categorized into two types: ① data dependency and ② row synchronization. First, data dependency arises when the MIN units attempt to access a block for the Q-memories and T-memories, but the SEL units have not yet updated it. Consequently, the MIN units must wait for the updated Q- and R-messages until the SEL units release this block. Second, our LDPC decoder employs row synchronization to manage the decoding schedule and simplify the control logic, which is beneficial to decode 5G NR LDPC codes with flexible code lengths and rates. However, this synchronization results in additional stalls if two consecutive layers have different row degrees. The decoding latency of our 5G NR LDPC decoder is presented in (13), where the bound is the summation of non-zero entries, I is the actual iteration number, and $\mathcal{D}_c$ is the delay of ① at the c-th layer.

$$\mathcal{L} = I \cdot \left(\underbrace{\sum_{c=0}^{M_p-1} d_c}_{\text{Bound}} + \underbrace{\sum_{c=0}^{M_p-1} \mathcal{D}_c}_{\text{stalls from ①}} + \underbrace{\sum_{c=0}^{M_p-1} \max\left(d_{c-1\oplus M_p} - d_c, 0\right)}_{\text{stalls from ②}} \right). \tag{13}$$

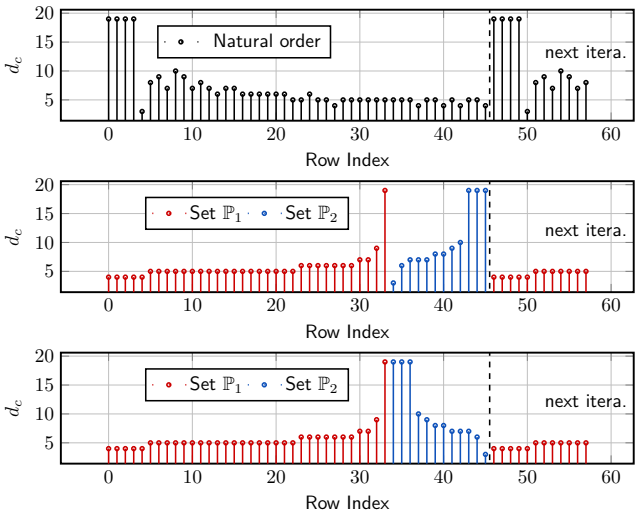


Fig. 11. Row-degree distributions of BG1 using various static schedule techniques (natural order, OSS 1-2, OSS 1-3).

E. Optimized Static Schedule (OSS)

In the following, we will optimize the schedule of the decoder to improve convergence and to reduce the number of stall cycles. Since the posterior LLRs are updated in a row-wise fashion, the convergence speed is greatly influenced by the order in which layers are processed. Generally, this order can be determined by either dynamic schedules [44] or static schedules [45], [46]. Static schedules offer a computational complexity advantage over dynamic schedules, as they do not require real-time calculations. Notably, some static schedule techniques are proposed in [45], [46] for 5G NR LDPC codes, but they ignore potential impact on throughput due to hardware constraints. In this section, we introduce a hardware-friendly OSS approach tailored to 5G NR. This OSS scheme delivers a 0.05 dB performance gain compared to conventional layered decoding and reduces the worst-case latency by around $15 \times I_{\max}$ cycles, compared to natural layer ordering.

First, we adopt two classical optimization principles to improve the error-correcting performance. In 5G, the first two columns of the base graphs are punctured to boost transmission efficiency. Let $\mathbb{P}_i$, $i \in \{0, 1, 2\}$, denote the sets of row indices in the base graphs with zero, one, and two punctured non-zero entries, respectively. The first optimization principle of our schedule dictates that we prioritize rows with fewer punctured non-zero entries. Subsequently, for the rows in the same $\mathbb{P}_i$, we decode the rows with smaller d_c first. These two optimization principles (least punctured and least row-degree) are also used in the BG based static schedule (BGSS) in [46] to speed up

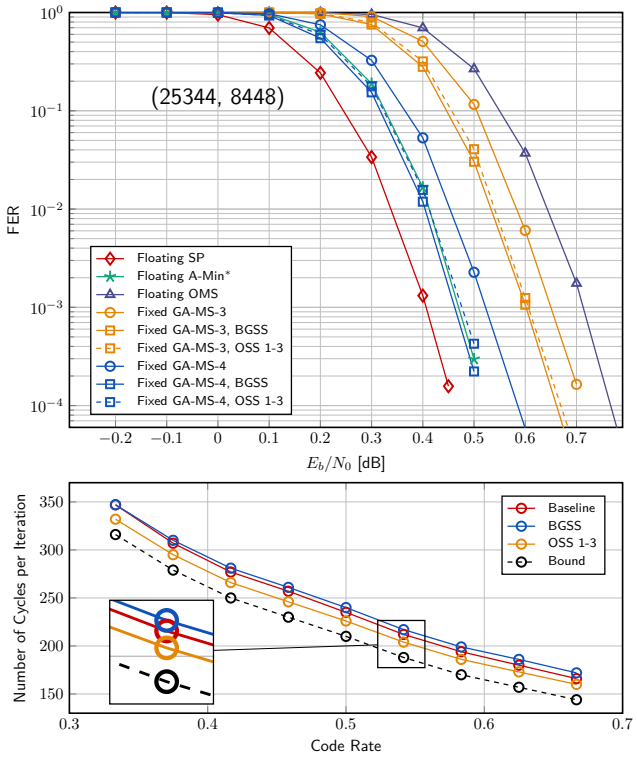


Fig. 12. FER performance and latency analysis for the proposed OSS scheme on BG1.

the decoding convergence.

The third optimization principle of our OSS scheme aims to diminish the worst-case latency in block-parallel architectures. As outlined in Section III-B, classical LDPC block-parallel decoders [24], [25] decouple (2) into several steps (e.g. the MIN and SEL phases) and execute them separately over different cycles. In most cases, the aforementioned steps can nearly overlap at two consecutive rows in the base graphs, i.e., each row can be processed within d_c cycles, which is also the latency bound of LDPC block-parallel decoders. However, possible stalls occur when two consecutive rows share column indices (data dependency) or have apparently higher row degrees (row synchronization), which is explained in detail in Section IV-D. Especially for the latter, stalls are unavoidable due to starvation of the pipeline. Hence, we arrange the rows of $\mathbb{P}_2$ in descending order of d_c to ensure that adjacent rows have preferably similar row degrees.

Fig. 11 displays the row-degree distributions of BG1 using various static schedule techniques. Note that the set $\mathbb{P}_0$ is empty in BG1. Based on OSS 1-2 (incorporating the first two principles, equivalent to BGSS in [46]), row-degree discontinuities appear at the junctions between $\mathbb{P}_1$ and $\mathbb{P}_2$, as well as between two consecutive iterations, resulting in redundant stalls in the LDPC decoder. However, OSS 1-3 (adopting all three principles) balance the error-rate and decoding latency. Since the majority of rows (74%) still adhere to the least punctured and least row-degree principles, OSS 1-3 features a fast decoding convergence. Then, by ordering the set $\mathbb{P}_2$ by descending d_c, we can minimize unnecessary stalls at the junctions of internal iterations. Importantly, pruning the columns or adjusting code rates only extends evenly on both

TABLE III
ASIC RESULTS OF THE PROPOSED 5G NR LDPC DECODER.

	This Work	
Technology [nm]	28	
Algorithm	GA-MS-3	
Iterations	4	
Voltage [V]	1.0	
Implementation	Synthesis	Post-layout
Core Area [mm^2]	1.274	1.823
Frequency [MHz]	1250	895
T/P† [Gbps]	$24.58^\wr$ 34.11^* $30.29^\star$ 34.00^+	$17.60^\wr$ 24.42^* $21.69^\star$ 24.34^+
Area Eff.† [Gbps/mm^2]	$19.29^\wr$ 26.77^* $23.78^\star$ 26.69^+	$9.66^\wr$ 13.40^* $11.90^\star$ 13.36^+

$^\wr$ BG1, $R=\frac{1}{3}$. * BG1 $R=\frac{8}{9}$. * BG2, $R=\frac{1}{5}$. $^+$ BG2, $R=\frac{2}{3}$.
† We set a fixed number of iterations to 4, without using early termination.

sides of the core rows (the first four rows with the maximum row-degree $d_c^{\max}$ in BG1 and BG2) and does not affect the property that adjacent rows have similar row degrees in OSS 1-3. Consequently, the OSS algorithm is compatible with all 5G NR LDPC codes.

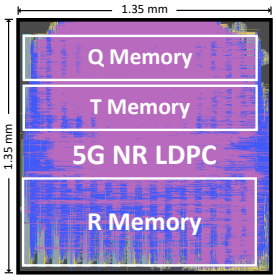
Fig. 12 illustrates that our OSS algorithm can yield a 0.05 dB improvement at FER $= 10^{-3}$. Fixed-point GA-MS-4 decoding with OSS only has a gap of 0.05 dB compared to floating-point SP decoding, and even outperforms floating-point A-Min* decoding before FER $= 10^{-3}$. Furthermore, we evaluate the number of cycles required per iteration using various static schedule schemes in Fig. 12. Our baseline is the conventional layered decoding [24]. The black dashed line represents the summation of non-zero entries in $\mathbf{H}_p$, serving as the lower bound on the number of cycles (no stalls) of a single iteration based on a block-parallel architecture. It is obvious that despite a 0.05 dB error-correcting improvement offered by BGSS, its row-degree discontinuities lead to increased stalls in a single iteration. In contrast, our OSS approach can reduce around 15 cycles per iteration, especially at low code rates, which is beneficial to alleviate the worst-case latency in practical communication scenarios.

Therefore, the aforementioned stalls in (13) can mostly be avoided through reasonable column reordering and the proposed OSS scheme. For stalls from ①, they can be removed by a simple column reordering [30]. Specifically, in two consecutive layers, we allow the MIN units to first visit independent blocks and let the SEL units visit dependent blocks, which can make most of $\mathcal{D}_c$ equal to 0, especially at low to medium code rates. For stalls from ②, following the least punctured and least row-degree principles, our OSS scheme can provide a layer reordering that has only one peak in the row-degree distribution. Therefore, the latency of our 5G NR LDPC decoder can be simplified as (14) from (13)

$$\mathcal{L} \approx I \cdot \left(\sum_{c=0}^{M_p-1} d_c + \left| d_c^{\max} - d_c^{\min} \right| \right). \tag{14}$$

V. Implementation Results

In this section, we present the implementation results of our 5G NR LDPC decoder based on a STM 28nm FD-SOI technology. The decoder is synthesized by Synopsys Design Compiler and placed and routed using Cadence Innovus Implementation System. Power analysis is done under typical operating conditions (1.0 V and 25 °C). To balance



Technology	28nm FD-SOI
Quantization [bit]	$(7, 5, 1)$
Core Area [mm^2]	1.35×1.35
Gate Count [M]	2.20
Voltage [V]	1.0
Frequency [MHz]	895
Peak T/P [Gbps]	24.42
Power [mW]	306.8
Energy [pJ/bit]	12.56

Fig. 13. A post-layout of the proposed 5G NR LDPC decoder implemented in a 28nm process, wherein the white boxes represent the integrated Q-memory, T-memory, and R-memory, implemented by 28nm FD-SOI dual-port SRAM.

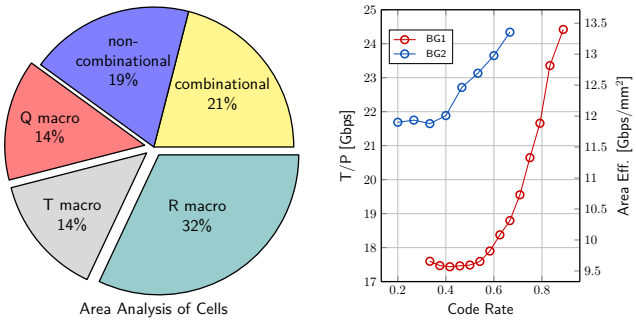


Fig. 14. Implementation results of our 5G NR LDPC decoder in a 28nm process across various code configurations and a fixed iteration value of 4.

error-correcting performance and hardware complexity, we employ the quantization scheme $(7, 5, 1)$ (as discussed in Section III-B) and incorporate fixed-point GA-MS-3 decoding into our decoder. All memory macros are based on STM 28nm FD-SOI dual-port SRAM. The size of the LUTs in the NCUs is 16×16 and each value in (11) is quantized as 4 bits. We instantiate 384 NCUs in the NCU pool to support the maximum lifting size in 5G. When $Z < Z_{\max}$, our decoder operates in a more fine-grained fashion by dividing into 16 groups. Each group is driven incrementally by different gating clocks, ensuring that block i $(i = 0, 1, \ldots, 15)$ is activated only if all other blocks j $(j = 0, 1, \ldots, i-1)$ are active. Based on the worst-case latency when using BG1 with $R = \frac{1}{3}$, the SEQ memory comprises 332 instruction words (in line with (14)), with each instruction being 59 bits in size.

A. Implementation Results for 5G NR LDPC Codes

Table III provides both the synthesis results and post-layout results of our 5G NR LDPC decoder. The synthesis results indicate that our decoder has a cell area of 1.274 mm^2 with a frequency of 1250 MHz. When all physical design processes (e.g., placement and routing) are done, the post-layout of our decoder has a core area of 1.823 mm^2 with a maximum operating frequency of 895 MHz. For LDPC codes at BG1 with $R = \frac{8}{9}$ and $Z = 384$, the implemented 5G NR LDPC decoder (setting a fixed iteration value of 4) achieves a peak throughput $\Theta_{\mathrm{T/P}}$ of 24.42 Gbps as follows:

$$\begin{aligned}\Theta_{\mathrm{T/P}} &= \frac{Z_{\max} \cdot N_p}{\mathcal{L}} \cdot F \\ &= \frac{384 \times 27}{380} \times 0.895 \text{ Gbps} = 24.42 \text{ Gbps},\end{aligned} \tag{15}$$

where $\mathcal{L} = 4 \times (79 + (19-3)) = 380$ cycles determined by (14) and F is the operating frequency. It is noteworthy that using the block-parallel architecture, our throughput $\Theta_{\mathrm{T/P}}$ already satisfies the peak throughput requirement of 20 Gbps as stipulated in the 5G standard [26].

TABLE IV
POWER BREAKDOWN† OF THE PROPOSED 5G NR LDPC DECODER.

	Power [mW]	**Percentage** [%]
Q-Memory	51.24	16.7
T-Memory	43.93	14.32
R-Memory	82.5	26.89
CSU	35.13	11.45
NCU	32.52	10.6
Controller	22.83	7.44
IO Interfaces	6.65	2.17
Others	32.0	10.43
Total	306.8	100

† We present the power breakdown in the case of $\Theta_{\mathrm{T/P}}$.

TABLE V
AVERAGE T/P OF PROPOSED DECODER WITH EARLY TERMINATION.

E_b/N_0 [dB]	3.0	3.5	4.0	4.5	5.0	5.5	6.0
Avg Iter.	15	11.15	6.05	4.40	3.49	2.79	2.35
Avg T/P [Gbps]	6.51	8.76	16.15	22.20	27.99	35.01	41.56

We use LDPC codes at BG1 with $R = \frac{8}{9}$ and $Z = 384$.

Fig. 13 illustrates the post-layout of our 5G NR LDPC decoder, where the core size is 1.35×1.35 mm^2 with a cell utilization of 72.9%. When running in the case of $\Theta_{\mathrm{T/P}}$, this decoder demonstrates a dynamic power of 306.8 mW and an energy consumption of 12.56 pJ/bit. Table IV presents a comprehensive power breakdown of our decoder. Notably, the power usage of the NCU logic only accounts for 10.6% of the total. In addition, Fig. 14 provides a detailed area analysis of cells. Memory macros account for 60% of the total cell area in our 5G NR LDPC decoder, with the Q-memory macros, T-memory macros, and R-memory macros contributing around 14%, 14%, and 32%, respectively. This substantial memory overhead in the 5G NR LDPC decoder mitigates the impact of complex decoding algorithms on overall hardware efficiency. Hence, within the context of achieving the peak throughput of 20 Gbps, using more complex decoding algorithms is justified to further enhance the error-correcting performance of 5G NR LDPC decoders.

As our decoder is compatible with all 5G NR LDPC codes, the corresponding throughput and area efficiency vary depending on the code configurations. With $Z_{\max} = 384$ and the maximum K_u, we sweep all code rates of BG1 $\left(\frac{1}{3} \leq R \leq \frac{8}{9}\right)$ and BG2 $\left(\frac{1}{5} \leq R \leq \frac{2}{3}\right)$ at a fixed iteration value of 4 and plot the corresponding throughput and area efficiency in Fig. 14. For BG1 with $R = \frac{1}{3}$, our decoder achieves a throughput of 17.60 Gbps and an area efficiency of 9.66 Gbps/mm^2. When the code rate increases to $\frac{8}{9}$, our decoder reaches a peak throughput of 24.42 Gbps and a maximum area efficiency of 13.40 Gbps/mm^2. BG2 exhibits a similar trend to BG1, with corresponding peak throughput and area efficiency values of 24.34 Gbps and 13.36 Gbps/mm^2, respectively. Moreover, our decoder can employ the PPCs as an early termination criterion to further enhance the average throughput. For LDPC codes at BG1 with $R = \frac{8}{9}$ and $Z = 384$, the average iteration and corresponding average throughput, under BPSK and $I_{\max} = 15$, are summarized in Table V.

TABLE VI
COMPARISONS WITH THE STATE-OF-THE-ART LDPC DECODERS.

	This work	TCAS-I'21 [15]$^\diamond$	ISCAS'21 [30]$^\diamond$	SSCL'22 [34]	TCAS-II'22 [33]$^\diamond$	TVT'23 [35]	TCAS-I'22 [31]	ASSCC'10 [25]	TVLSI'15 [47]	JSSC'10 [23]
Technology [nm]	28	90	28	40	65	90	65	90	90	65
Algorithm	GA-MS-3	IAMS	NMS	NMS	MS	SOMS	OMS	OMS	OMS	OMS
Implementation	Post-layout	Synthesis	Post-layout	Silicon	Synthesis	Post-layout	Post-layout	Silicon	Post-layout	Silicon
Voltage [V]	1.0	–	–	0.9	–	1.0	1.2	1.0	0.9	0.7
Standard	5G NR	5G NR	5G NR	5G NR	5G NR	5G NR	5G NR	802.11n	802.11n	10GBASE-T
Architecture	block	row	block	row	row	row	partial	block	block	partial
Iterations	4	15	1	5	10	10	3	10	10	8
Max Code Length	26112	2600	26112	6400	1664	3072	26112	1944	1944	2048
Frequency [MHz]	895	158.2	556	180	244	192.3	500	346	336	100
Area [mm^2]	1.823	1.353	1.97	2.07	1.16	6.45	5.74	1.77	5.2	5.35
Gate Count [M]	2.20	0.24	2.83	1.69	0.81	–	2.67	–	0.51	–
Peak T/P* [Gbps]	24.42	0.914	33.2	2.29	4.1	9.6	21.78	0.679	1.71	2.13
Power [mW]	306.8	76.4	232	139.4	115.8	3456	413	107.3	451.3	144
Scaled to 28nm, 1.0 V†, and a fixed iteration value of 4‡										
Area [mm^2]	1.823	**0.131**	1.97	1.014	0.215	0.624	1.065	0.171	0.503	0.993
Peak T/P [Gbps]	24.42	11.02	8.3	4.09	23.79	**77.14**	37.92	5.46	13.74	9.89
Area Eff. [Gbps/mm^2]	13.40	84.10	4.21	4.03	110.67	**123.63**	35.61	31.91	27.32	9.96
Power [mW]	306.8	**23.77**	232	120.47	49.88	1075.2	123.55	33.38	173.34	126.59
Energy [pJ/bit]	12.56	2.16	27.95	29.45	**2.10**	13.94	3.26	6.11	12.62	12.80

† Scaled to 28nm and 1.0 V with area $\propto s^2$, frequency $\propto 1/s$, and power $\propto s \cdot u^2$, where s is the scaling factor to 28nm and u is the scaling factor to 1.0 V.
‡ We let the number of iterations be fixed to focus on the worst-case performance.
$^\diamond$ For the missing voltages in [15], [30], [33], we assume these works all operate at 1.0 V.
* In our 5G NR LDPC decoder, the peak throughput is attained with a code configuration of BG1, $R = \frac{8}{9}$, and $Z = 384$.

B. Comparison With Previous Works

Table VI provides a detailed comparison between our 5G NR LDPC decoder with the SOA decoder implementations in [15], [23], [25], [30], [31], [33]–[35], [47]. To ensure fairness, we normalize all previous works to a 28nm process with a supply voltage of 1.0 V and set a fixed number of iterations to 4. Note that there is no early termination in Table VI to focus on the architecture. Compared to a similar block-parallel 5G NR LDPC decoder in [30], our work has a 2.94× peak throughput, a 3.18× area efficiency, and 55% less energy consumption. When compared to the SOA row-parallel architectures presented in [15], [33]–[35], our decoder achieves a throughput that is 2.22× faster than [15] and 1.03× faster than [33]. Moreover, it demonstrates 3.32× greater area efficiency than [34] and consumes 9.9% less energy than [35]. Although the area overhead of these row-parallel 5G NR LDPC decoders [15], [33]–[35] is better than our results, their maximum code lengths are much shorter than $N = 26112$ required by the 5G standard, granting them a significant area advantage. Indeed, these row-parallel architectures will suffer from high routing complexity to be compatible with all 5G NR LDPC codes. In comparison with the 5G NR LDPC decoder in [31], our peak throughput is 35.6% inferior to [31], but the PRP architecture of [31] has long decoding latency at medium to low code rates. For instance, for LDPC codes at BG1 with $R = \frac{1}{3}$ and $Z = 384$, our decoder can yield 17.60 Gbps at a fixed iteration value of 4, but the normalized throughput of [31] is only 9.74 Gbps (calculated by (5) in [31]). It is noteworthy that our GA-MS-3 decoder has a lower error-rate than the OMS decoder in [31], as shown in Fig. 6. Given that the area overhead (e.g., routing and storage complexity) of LDPC decoders is largely influenced by varying maximum code lengths that can be processed and therefore difficult to compare, we only plot the peak throughput against energy in Fig. 15 for various SOA LDPC decoders listed in Table VI. From Fig. 15, our decoder outperforms many existing decoders

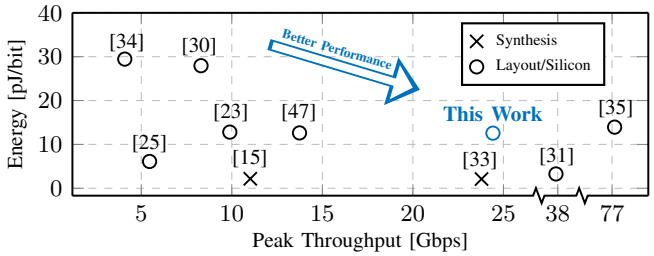


Fig. 15. Peak throughput vs. energy (normalized to 28nm) of various SOA LDPC decoders.

but is still inferior to [31]. Nonetheless, as mentioned before, our decoder achieves higher throughput than [31] at medium to low code rates and maintains a lower error-rate. In conclusion, our decoder has an energy of 12.56 pJ/bit, consuming 55%, 57.4%, and 9.9% less than [30], [34], [35], respectively. This work also achieves a peak throughput of 24.42 Gbps which is 2.22×, 2.94×, 5.97×, 1.03×, 4.47×, 1.78×, and 2.47× faster than the SOA LDPC decoders [15], [30], [34], [33], [25], [47], [23]. Moreover, the maximum area efficiency in our 5G NR decoder is 13.40 Gbps/mm^2, which is 3.18×, 3.32×, and 1.35× higher than [30], [34], [23], respectively.

VI. CONCLUSIONS

In this paper, we propose high-performance and low-complexity GA-MS decoding. By truncating the number of incoming messages in CN processing, we can make a trade-off between error-correcting performance and computational complexity. By incorporating the well-designed LUTs, quantization schemes, and other approximation techniques, our fixed-point GA-MS decoding exhibits only a minor gap of 0.1 dB compared to floating-point SP decoding under various 5G NR code configurations and high-order modulations. We also present a fully reconfigurable 5G NR LDPC decoder implementation, compatible with all 5G NR LDPC codes. The

28nm FD-SOI post-layout implementation results show that our decoder has a core area of 1.823 mm^2, achieves a peak throughput of 24.42 Gbps at 895 MHz, and has an energy consumption of 12.56 pJ/bit with a supply voltage of 1.0 V.

REFERENCES

[1] R. Gallager, "Low-density parity-check codes," *IRE Trans. Inf. Theory*, vol. 8, no. 1, pp. 21–28, Jan. 1962.

[2] *Standard: Synchronization standard for distributed transmission*, Advanced Television System Committee (ATSC), Feb. 2007.

[3] *Wireless LAN medium access control (MAC) and physical layer (PHY) specifications: Enhancements for higher throughput*, IEEE P802.11n/D5.02, Part 11, Jul. 2008.

[4] *Digital video broadcasting (DVB) user guidelines for the second generation system for broadcasting, interactive services, news gathering and other broadband satellite applications (DVB-S2)*, ETSI TR 102 376, Feb. 2009.

[5] *Chairman's notes of AI 7.1.5 on consideration on LDPC design for NR*, 3GPP R1-1611112 Release 16, Nov. 2016.

[6] *5G NR: multiplexing and channel coding*, 3GPP TS 38.212 version 15.2.0 Release 15, Jul. 2018.

[7] F. Kschischang, B. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.

[8] T. Richardson and R. Urbanke, "The capacity of low-density parity-check codes under message-passing decoding," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 599–618, Feb. 2001.

[9] M. Mansour and N. Shanbhag, "High-throughput LDPC decoders," *IEEE Trans. VLSI Syst.*, vol. 11, no. 6, pp. 976–996, Dec. 2003.

[10] N. Wiberg, "Codes and decoding on general graphs," 1996.

[11] M. P. Fossorier, M. Mihaljevic, and H. Imai, "Reduced complexity iterative decoding of low-density parity check codes based on belief propagation," *IEEE Trans. Commun.*, vol. 47, no. 5, pp. 673–680, May 1999.

[12] J. Chen, A. Dholakia, E. Eleftheriou, M. Fossorier, and X.-Y. Hu, "Reduced-complexity decoding of LDPC codes," *IEEE Trans. Commun.*, vol. 53, no. 8, pp. 1288–1299, Aug. 2005.

[13] X. Wu, Y. Song, M. Jiang, and C. Zhao, "Adaptive-normalized/offset min-sum algorithm," *IEEE Commun. Lett.*, vol. 14, no. 7, pp. 667–669, Jul. 2010.

[14] K. Le Trung, F. Ghaffari, and D. Declercq, "An adaptation of min-sum decoder for 5G low-density parity-check codes," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2019, pp. 1–5.

[15] H. Cui, F. Ghaffari, K. Le, D. Declercq, J. Lin, and Z. Wang, "Design of high-performance and area-efficient decoder for 5G LDPC codes," *IEEE Trans. Circuits Syst. I*, vol. 68, no. 2, pp. 879–891, Feb. 2020.

[16] V. Savin, "Self-corrected min-sum decoding of LDPC codes," in *Proc. IEEE Int. Symp. Inf. Theory*, 2008, pp. 146–150.

[17] J. Zhang, M. Fossorier, and D. Gu, "Two-dimensional correction for min-sum decoding of irregular LDPC codes," *IEEE Commun. Lett.*, vol. 10, no. 3, pp. 180–182, Mar. 2006.

[18] P. Kang, Y. Xie, L. Yang, and J. Yuan, "Enhanced quasi-maximum likelihood decoding based on 2D modified min-sum algorithm for 5G LDPC codes," *IEEE Trans. Commun.*, vol. 68, no. 11, pp. 6669–6682, Nov. 2020.

[19] C. Jones, E. Valles, M. Smith, and J. Villasenor, "Approximate-min constraint node updating for LDPC code decoding," in *Proc. IEEE Military Comm. Conf.*, vol. 1, 2003, pp. 157–162.

[20] W. Zhou and M. Lentmaier, "Generalized two-magnitude check node updating with self correction for 5G LDPC codes decoding," in *Proc. IEEE Int. Conf. on Syst. Comm. Coding*, 2019, pp. 1–6.

[21] T. J. Richardson, S. Kudekar, and V. Loncke, *Adjusted mim-sum decoder*, US Patent, Apr. 2017.

[22] T.-C. Kuo and A. N. Willson, "A flexible decoder IC for WiMAX QC-LDPC codes," in *Proc. IEEE Custom Integrated Circuits Conf.*, 2008, pp. 527–530.

[23] Z. Zhang, V. Anantharam, M. J. Wainwright, and B. Nikolic, "An efficient 10GBASE-T Ethernet LDPC decoder design with low error floors," *IEEE J. Solid-State Circuits*, vol. 45, no. 4, pp. 843–855, Apr. 2010.

[24] C. Studer, N. Preyss, C. Roth, and A. Burg, "Configurable high-throughput decoder architecture for quasi-cyclic LDPC codes," in *Proc. IEEE Asilomar Conf. Signals, Syst. Compt.*, 2008, pp. 1137–1142.

[25] C. Roth, P. Meinerzhagen, C. Studer, and A. Burg, "A 15.8 pJ/bit/iter quasi-cyclic LDPC decoder for IEEE 802.11n in 90 nm CMOS," in *Proc. IEEE Asian Solid-State Circuits Conf.*, 2010, pp. 1–4.

[26] D. Hui, S. Sandberg, Y. Blankenship, M. Andersson, and L. Grosjean, "Channel coding in 5G new radio: A tutorial overview and performance comparison with 4G LTE," *IEEE Veh. Technol. Mag.*, vol. 13, no. 4, pp. 60–69, Dec. 2018.

[27] A. J. Blanksby and C. J. Howland, "A 690-mW 1-Gb/s 1024-b, rate-1/2 low-density parity-check code decoder," *IEEE J. Solid-State Circuits*, vol. 37, no. 3, pp. 404–412, Mar. 2002.

[28] C.-C. Cheng, J.-D. Yang, H.-C. Lee, C.-H. Yang, and Y.-L. Ueng, "A fully parallel LDPC decoder architecture using probabilistic min-sum algorithm for high-throughput applications," *IEEE Trans. VLSI Syst.*, vol. 61, no. 9, pp. 2738–2746, Sep. 2014.

[29] R. Ghanaatian, A. Balatsoukas-Stimming, T. C. Müller, M. Meidlinger, G. Matz, A. Teman, and A. Burg, "A 588-Gb/s LDPC decoder based on finite-alphabet message passing," *IEEE Trans. VLSI Syst.*, vol. 26, no. 2, pp. 329–340, Feb. 2018.

[30] C.-Y. Lin, L.-W. Liu, Y.-C. Liao, and H.-C. Chang, "A 33.2 Gbps/iter. reconfigurable LDPC decoder fully compliant with 5G NR applications," in *Proc. IEEE Int. Symp. Circuits Syst.*, 2021, pp. 1–5.

[31] S. Lee, S. Park, B. Jang, and I.-C. Park, "Multi-mode QC-LDPC decoding architecture with novel memory access scheduling for 5G New-Radio standard," *IEEE Trans. Circuits Syst. I*, vol. 69, no. 5, pp. 2035–2048, May 2022.

[32] J. Nadal and A. Baghdadi, "Parallel and flexible 5G LDPC decoder architecture targeting FPGA," *IEEE Trans. VLSI Syst.*, vol. 29, no. 6, pp. 1141–1151, Jun. 2021.

[33] S. Yun, B. Y. Kong, and Y. Lee, "Area-and energy-efficient LDPC decoder using mixed-resolution check-node processing," *IEEE Trans. Circuits Syst. II*, vol. 69, no. 3, pp. 999–1003, Mar. 2022.

[34] B.-S. Su, C.-H. Lee, and T.-D. Chiueh, "A 58.6/91.3 pJ/b dual-mode belief-propagation decoder for LDPC and polar codes in the 5G communications standard," *IEEE Solid-State Circuits Lett.*, vol. 5, Apr. 2022.

[35] A. Verma and R. Shrestha, "Low computational-complexity SOMS-algorithm and high-throughput decoder architecture for QC-LDPC codes," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 66–80, Jan. 2023.

[36] R. Tanner, "A recursive approach to low complexity codes," *IEEE Trans. Inf. Theory*, vol. 27, no. 5, pp. 533–547, Sep. 1981.

[37] M. P. Fossorier, "Quasi-cyclic low-density parity-check codes from circulant permutation matrices," *IEEE Trans. Inf. Theory*, vol. 50, no. 8, pp. 1788–1793, Aug. 2004.

[38] H. Zhong and T. Zhang, "Block-LDPC: A practical LDPC coding system design approach," *IEEE Trans. Circuits Syst. I*, vol. 52, no. 4, pp. 766–775, Apr. 2005.

[39] Z. Zhong, Y. Huang, Z. Zhang, X. You, and C. Zhang, "A flexible and high parallel permutation network for 5G LDPC decoders," *IEEE Trans. Circuits Syst. II*, vol. 67, no. 12, pp. 3018–3022, Jun. 2020.

[40] E. Sharon, S. Litsyn, and J. Goldberger, "An efficient message-passing schedule for LDPC decoding," in *Proc. IEEE Conven. Elect. Electron. Eng. Isreal*, 2004, pp. 223–226.

[41] D. E. Hocevar, "A reduced complexity decoder architecture via layered decoding of LDPC codes," in *Proc. IEEE Workshop Signal Process. Syst.*, 2004, pp. 107–112.

[42] J. Mao, M. A. Abdullahi, P. Xiao, and A. Cao, "A low complexity 256QAM soft demapper for 5G mobile system," in *Proc. IEEE Euro. Conf. Netw. Commun.*, 2016, pp. 16–21.

[43] B. Le Gal, Y. Delomier, C. Leroux, and C. Jégo, "Low-latency sorter architecture for polar codes successive-cancellation-list decoding," in *Proc. IEEE Workshop Signal Process. Syst.*, 2020, pp. 1–5.

[44] T. C.-Y. Chang, P.-H. Wang, J.-J. Weng, I.-H. Lee, and Y. T. Su, "Belief-propagation decoding of LDPC codes with variable node–centric dynamic schedules," *IEEE Trans. Commun.*, vol. 69, no. 8, pp. 5014–5027, Aug. 2021.

[45] C.-Y. Liang, M.-R. Li, H.-C. Lee, H.-Y. Lee, and Y.-L. Ueng, "Hardware-friendly LDPC decoding scheduling for 5G HARQ applications," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2019, pp. 1418–1422.

[46] K. Tian and H. Wang, "A novel base graph based static scheduling scheme for layered decoding of 5G LDPC codes," *IEEE Commun. Lett.*, vol. 26, no. 7, pp. 1450–1453, Jul. 2022.

[47] S. Kumawat, R. Shrestha, N. Daga, and R. Paily, "High-throughput LDPC-decoder architecture using efficient comparison techniques & dynamic multi-frame processing schedule," *IEEE Trans. Circuits Syst. I*, vol. 62, no. 5, pp. 1421–1430, May 2015.

Yuqing Ren (Student Member, IEEE) was born in 1996. He received the B.S. degree and M.E. degree from Nanjing University and Science and Technology and Southeast University, Nanjing, China, in 2018 and 2021, respectively. He is currently pursuing a Ph.D. degree in Electrical Engineering under the supervision of Prof. Andreas Burg in the Telecommunication Circuit Laboratory (TCL) at École Polytechnique Fédérale de Lausanne (EPFL), Switzerland. His research interests include error correction coding theory and practice, digital signal processing, beyond 5G and 6G, and VLSI circuits for communications.

Hassan Harb (Member, IEEE) currently serves as an FPGA Engineer within the Research and Development department at Leica Geosystems, Hexagon, located in Heerbrugg, Switzerland. He obtained his Bachelor's degree and completed his Master's degree in Communications and Electronics in 2015 from the Lebanese University. Subsequently, he earned his Ph.D. in NB-LDPC codes construction and efficient NB-LDPC decoder hardware design in 2018 from L'université Bretagne-Sud, Lorient, France. Following the completion of his Ph.D., he worked as a Post-doctoral Researcher until February 2022, where he made significant contributions to the fields of Turbo and LDPC codes, including code construction, decoding algorithms, and efficient hardware designs.

Yifei Shen (Member, IEEE) was born in 1997. He received the B.S. degree from the Chien-Shiung Wu College (Honors College) in 2016, and the M.S. and Ph.D. degrees from the School of Information Science and Engineering, Southeast University, Nanjing, China, in 2018 and 2022, respectively. He is currently a Postdoctoral Researcher with the Telecommunication Circuits Laboratory, École Polytechnique Fédérale de Lausanne (EPFL), Switzerland. His research interests include error-correcting codes, VLSI design for digital signal processing, and synthetic biology. He was the recipient of the Best Student Paper Award at the 2016 IEEE International Conference on Digital Signal Processing, the 2020 IEEE Circuits and Systems Society Pre-Doctoral Scholarship Award, and the 2023 Chinese Institute of Electronics Best Doctoral Thesis Award.

Alexios Balatsoukas-Stimming (Member, IEEE) received the Diploma and M.Sc. degrees in electronics and computer engineering from the Technical University of Crete, Chania, Greece, in 2010 and 2012, respectively, and the Ph.D. degree in computer and communications sciences from the Ecole polytechnique fédérale de Lausanne (EPFL), Switzerland, in 2016. He was a Marie Skłodowska-Curie Post-doctoral Fellow with the European Laboratory for Particle Physics, Meyrin, Switzerland, for one year. He was a Post-doctoral Researcher with the Telecommunications Circuits Laboratory, EPFL from 2018 to 2019. He has been a Visiting Post-doctoral Researcher with Cornell University, Ithaca, NY, USA, and the University of California at Irvine, Irvine, CA, USA. He is currently an Assistant Professor with the Department of Electrical Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands. His research interests include VLSI circuits for communications, error correction coding theory and practice, and the applications of approximate computing and machine learning to signal processing for communications.

Andreas Burg (Senior Member, IEEE) was born in Munich, Germany, in 1975. He received his Dipl.-Ing. degree from the Swiss Federal Institute of Technology (ETH) Zurich, Zurich, Switzerland, in 2000, and the Dr. sc. techn. degree from the Integrated Systems Laboratory of ETH Zurich, in 2006. In 1998, he worked at Siemens Semiconductors, San Jose, CA. During his doctoral studies, he worked at Bell Labs Wireless Research for a total of one year. From 2006 to 2007, he was a postdoctoral researcher at the Integrated Systems Laboratory and at the Communication Theory Group of the ETH Zurich. In 2007 he co-founded Celestrius, an ETH-spinoff in the field of MIMO wireless communication, where he was responsible for the ASIC development as Director for VLSI. In January 2009, he joined ETH Zurich as SNF Assistant Professor and as head of the Signal Processing Circuits and Systems group at the Integrated Systems Laboratory. In January 2011, he joined the École Polytechnique Fédérale de Lausanne (EPFL) where he is leading the Telecommunications Circuits Laboratory. He was promoted to Associate Professor with Tenure in June 2018. In 2021, he co-founded RAAAM Memory Technologies, a spinoff from EPFL and Bar-Ilan University, to commercialize the highest density embedded memories in any standard CMOS technology.

Mr. Burg has served on the TPC of various conferences on signal processing, communications, and VLSI. He was a TPC co-chair for VLSI-SoC 2012 and the TCP co-chair for ESSCIRC 2016 and SiPS 2017. He was a General Chair of ISLPED 2019 and he served as an Editor for the IEEE TRANSACTION OF CIRCUITS AND SYSTEMS in 2013 and has is currently an editor of the IEEE TRANSACTIONS ON VLSI and the IEEE TRANSACTIONS ON SIGNAL PROCESSING.